SMART AGRICULTURE & EDGE AI MOBILE SERIES

เอกสารคำสอนและคู่มือปฏิบัติการขั้นสูง • มหาวิทยาลัยราชภัฏรำไพพรรณี

การออกแบบและพัฒนาแอปพลิเคชันสมาร์ทโฟน วิเคราะห์โรคในทุเรียน

[สำหรับผู้เริ่มต้นไม่มีพื้นฐานการเขียนโค้ด]

DESIGN AND DEVELOPMENT OF SMARTPHONE APPLICATION FOR DURIAN DISEASE ANALYSIS

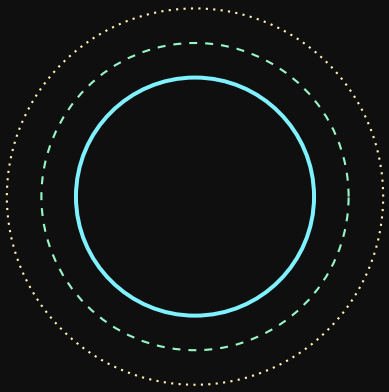
(A NO-CODE & BEGINNER-FRIENDLY GUIDE TO COMPUTER VISION & EDGE AI)



ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาปัญญาประดิษฐ์เพื่อเกษตรดิจิทัลและสิ่งแวดล้อม คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏรำไพพรรณี • พ.ศ. 2569



SMART AGRICULTURE & EDGE AI MOBILE SERIES

การออกแบบและพัฒนาแอปพลิเคชันสมาร์ทโฟน
วิเคราะห์โรคในทุเรียน

[สำหรับผู้เริ่มต้นไม่มีพื้นฐานการเขียนโค้ด]

DESIGN AND DEVELOPMENT OF SMARTPHONE APPLICATION

FOR DURIAN DISEASE ANALYSIS

(A NO-CODE & BEGINNER-FRIENDLY GUIDE TO COMPUTER VISION & EDGE AI)

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาปัญญาประดิษฐ์เพื่อเกษตรดิจิทัลและสิ่งแวดล้อม

คณะวิทยาศาสตร์และเทคโนโลยี

มหาวิทยาลัยราชภัฏรำไพพรรณี

คำนำ

เอกสาร คำ สอน และ ตำรา วิชาการ ฉบับ นี้ จัด ทำ ขึ้น เพื่อ รวบรวม องค์ ความ รู้ ด้าน กา รบู รณา การ เทคโนโลยี คอมพิวเตอร์วิทัศน์ (Computer Vision) และการเรียนรู้เชิง ลึก (Deep Learning) เข้า กับ งาน พยาธิ วิทยา พืช (Plant Pathology) โดยมุ่งเน้นการแก้ปัญหาโรคใบทุเรียนซึ่งเป็น พืชเศรษฐกิจหลักของภาคตะวันออกและประเทศไทยอย่าง เป็นระบบ

เนื้อหาในเล่มได้รับการปรับปรุงและจัดแบ่งออกเป็น 7 บทเรียนสมบูรณ์ ครอบคลุมตั้งแต่:

1. วิทยาการ โรค ใบ ทุเรียน และ สถาปัตยกรรม เกษตร อัจฉริยะ (Edge AI Paradigm)
2. การประมวลผลภาพดิจิทัลและการรวบรวมเตรียมชุด ข้อมูลใบทุเรียน
3. สถาปัตยกรรมโครงข่ายประสาทเทียมคอนโวลูชันและ การเรียนรู้แบบถ่ายโอน (MobileNetV3)
4. การ ฝึกฝน และ การ ประเมิน ประสิทธิภาพ แบบ จำลอง ด้วย PyTorch
5. ปัญญา ประดิษฐ์ อธิบาย ได้ (Explainable AI: Grad-CAM) และการตรวจหาตำแหน่งรอยโรค

6. การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI (ONNX & Quantization INT8)
7. การพัฒนาแอปพลิเคชันสมาร์ทโฟน (Flutter) และการประยุกต์ใช้งานจริงในแปลงเกษตร

จุดเด่นของตำราเล่มนี้คือการออกแบบกระบวนการเรียนรู้แบบ No-Code & Beginner-Friendly ควบคู่ไปกับโค้ดภาษา Python และ PyTorch เชิงลึก ช่วยให้ผู้เริ่มต้นและผู้ที่ไม่มีพื้นฐานการเขียนโปรแกรม สามารถสร้าง พัฒนา และประยุกต์ใช้โมเดลปัญญาประดิษฐ์ตรวจโรคพืชบนสมาร์ทโฟนได้จริง พร้อมระบบ Explainable AI (Grad-CAM) ที่แสดงภาพความร้อนชี้ตำแหน่งรอยโรคเพื่อยืนยันความถูกต้องทางพยาธิวิทยา

ผู้เขียนหวังเป็นอย่างยิ่งว่าตำราเล่มนี้จะเป็นประโยชน์ต่อนักศึกษาเกษตรกร ชาวสวนทุเรียน นักวิจัย และผู้สนใจในการพัฒนานวัตกรรมเกษตรอัจฉริยะอย่างยั่งยืน

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา
มหาวิทยาลัยราชภัฏรำไพพรรณี
พ.ศ. 2569

Contents

คำนำ i

1 วิทยาการ โรค ใบ ทุเรียน และ สถาปัตยกรรม เกษตร อัจฉริยะ 1
แผนบริหารการสอนประจำบทที่ 1 1
1.1 บริบท ทาง เศรษฐกิจ และ ปัญหา โรค ใบ ทุเรียนในประเทศไทย 4
1.2 ชีววิทยาและการระบาดของเชื้อสาเหตุทางใบ 6
1.2.1 โรคใบจุดสาหร่าย (Algal Leaf Spot) 6
1.2.2 โรคใบติด (Leaf Blight / Rhizoctonia Blight) 6
1.2.3 โรคใบจุดสนิมและแอนแทรคโนส (Leaf Spot / Anthracnose) . . . 7
1.3 สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และ การตรวจวินิจฉัยด้วย Edge AI 7
1.4 ปฏิทินการระบาดตามฤดูกาลและมาตรการเฝ้าระวังเชิงรุก 8
1.5 ความ ท้าทาย ใน การ ตรวจ จับ รอย โรค ใน สภาพแปลงเปิด 10
1.6 โค้ด ตัวอย่าง: สคริปต์วิเคราะห์ความเสี่ยงการระบาดตามสภาพอากาศ 11

1.7 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 13
คำถามทบทวนท้ายบทที่ 1 14
กิจกรรมฝึกปฏิบัติการประจำบทที่ 1 16

2 การประมวลผลภาพดิจิทัลและการเตรียมชุดข้อมูลใบทุเรียน 17
แผนบริหารการสอนประจำบทที่ 2 17
2.1 พื้นฐานการแทนค่าภาพดิจิทัลและปริภูมิสี . 20
2.2 การสำรวจและการกระจายตัวของชุดข้อมูลใบทุเรียน 21
2.3 เทคนิคการเพิ่มพูนข้อมูลภาพ (Data Augmentation) 22
2.4 การสร้าง Custom PyTorch Dataset และ Data Pipeline 25
2.5 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 28
คำถามทบทวนท้ายบทที่ 2 29
กิจกรรมฝึกปฏิบัติการประจำบทที่ 2 30

3 สถาปัตยกรรมโครงข่ายประสาทเทียมและการเรียนรู้แบบถ่ายโอน 33
แผนบริหารการสอนประจำบทที่ 3 33
3.1 พื้นฐานการแปลงคอนโวลูชันและการสกัดคุณลักษณะภาพ 37
3.2 สถาปัตยกรรม MobileNetV3 และ Depthwise Separable Convolution 37
3.3 บล็อก Inverted Residual และ Squeeze-and-Excitation Module 39
3.4 การ เปรียบ เทียบ สถาปัตยกรรม โมเดลสำหรับงาน Edge AI 40

3.5 ทฤษฎี Transfer Learning และ การ ปรับแต่ง Classifier Head 41
3.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 44
คำถามทบทวนท้ายบทที่ 3 45
กิจกรรมฝึกปฏิบัติการประจำบทที่ 3 46

4 การฝึกฝนและการประเมินประสิทธิภาพแบบจำลองด้วย PyTorch 49
แผนบริหารการสอนประจำบทที่ 4 49
4.1 ฟังก์ชัน การ สูญ เสีย และ ขั้น ตอน วิธี ปรับพารามิเตอร์ 53
4.2 การ วิเคราะห์ ผลลัพธ์ การ ฝึกฝน และ เส้น กราฟการเรียนรู้ 54
4.3 การประเมินผลเชิงลึกด้วย Confusion Matrix 55
4.4 การบันทึกภาพหน้าจอขณะรันโค้ดในเทอร์มินัลจริง 56
4.5 โค้ดตัวอย่าง: ลูปการฝึกฝนและประเมินผลฉบับสมบูรณ์ 57
4.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 60
คำถามทบทวนท้ายบทที่ 4 62
กิจกรรมฝึกปฏิบัติการประจำบทที่ 4 63

5 ปัญญา ประดิษฐ์ อธิบาย ได้ และ การ ชี้ ตำแหน่ง รอย โรคด้วย Grad-CAM 65
แผนบริหารการสอนประจำบทที่ 5 65
5.1 ความ สำคัญ ของ Explainable AI ใน งาน วิทยาศาสตร์การเกษตร 69
5.2 ทฤษฎีและสมการคณิตศาสตร์ของ Grad-CAM 71
5.3 การวิเคราะห์ภาพผลลัพธ์ Grad-CAM ของโรคใบทุเรียน 73

5.4 การ ประเมิน ระดับ ความ รุนแรง ของ โรค (Severity Index Estimation) 74
5.5 โค้ด ตัวอย่าง: คลาส GradCAM ฉบับ สมบูรณ์ใน PyTorch 76
5.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 78
คำถามทบทวนท้ายบทที่ 5 79
กิจกรรมฝึกปฏิบัติการประจำบทที่ 5 81

6 การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI 83
แผนบริหารการสอนประจำบทที่ 6 83
6.1 ข้อ จำกัด ของ อุปกรณ์ ประมวล ผล บน ขอบ และแนวทางการปรับแต่ง 87
6.2 มาตรฐานเปิด ONNX และ ONNX Runtime Engine 87
6.3 ทฤษฎี การ ลด ทอน ความ แม่นยำ ตัวเลข (Quantization INT8) 88
6.4 การเปรียบเทียบประสิทธิภาพบนฮาร์ดแวร์ประมวลผลจริง 90
6.5 โค้ด ตัวอย่าง: สคริปต์ ส่ง ออก ONNX และ ตรวจสอบ Parity 91
6.6 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 94
คำถามทบทวนท้ายบทที่ 6 95
กิจกรรมฝึกปฏิบัติการประจำบทที่ 6 96

7 การ พัฒนา แอป พลิ เค ชัน สมา ร์ท โฟน และ การ ประยุกต์ใช้งานจริง 99
แผนบริหารการสอนประจำบทที่ 7 99
7.1 สถาปัตยกรรมแอปพลิเคชันมือถือ Flutter Edge AI 103

7.2 การออกแบบส่วนต่อประสานผู้ใช้และการวินิจฉัยแบบสด 104
7.3 การ ประเมิน การ ยอมรับ เทคโนโลยี ของเกษตรกร (TAM Evaluation) 105
7.4 โค้ด ตัวอย่าง: สถาปัตยกรรม Inference Controller ใน Flutter 107
7.5 ตัวอย่างการคำนวณและตัวอย่างการนำไปใช้งานจริง 109
คำถามทบทวนท้ายบทที่ 7 111
กิจกรรมฝึกปฏิบัติการประจำบทที่ 7 112

A คู่มือการติดตั้งและตั้งค่าสภาพแวดล้อมการทำงาน 115
A.1 การเตรียมสภาพแวดล้อมบน Google Colab (Cloud GPU) 115
A.2 การ ติด ตั้ง สภาพ แวดล้อม บน เครื่อง คอมพิวเตอร์ส่วนบุคคล (Local Machine) . 116
A.3 การ ติด ตั้ง ชุด พัฒนา Flutter สำหรับ การสร้างแอปพลิเคชันมือถือ 117

B ตารางค่าพารามิเตอร์ของแบบจำลองและการปรับแต่ง 119
B.1 ตา ราง ค่า ไฮ เปอร์ พารามิเตอร์ สำหรับ การฝึกฝนโมเดล (Training Configurations) . 120
B.2 ตาราง โครงสร้าง เลเยอร์ ของ โมเดล MobileNetV3-Small 121

C พจนานุกรมคำศัพท์เทคนิคและนิยามศัพท์ 123

บรรณานุกรม 127

ดรรชนีสืบค้น 131

ประวัติผู้เขียน 133

List of Tables

1.1 กลุ่มโรคใบทุเรียนสำคัญ เชื้อสาเหตุ และผลกระทบต่อผลผลิตทางการเกษตร 5
1.2 ปฏิทินระบาดวิทยาของโรคใบทุเรียนและระดับความเสี่ยงในรอบปีการผลิตภาคตะวันออก 9

2.1 พารามิเตอร์ของเทคนิคการเพิ่มพูนข้อมูลภาพที่ใช้ในไปป์ไลน์ PyTorch 24

3.1 การเปรียบเทียบคุณลักษณะทางเทคนิคของสถาปัตยกรรมโครงข่ายประสาทเทียมสำหรับอุปกรณ์ขอบ 41

4.1 ตารางสรุปตัวชี้วัดประสิทธิภาพเชิงปริมาณจำแนกตามรายคลาสโรคใบทุเรียน 56

5.1 การเปรียบเทียบคุณสมบัติของเทคนิค Explainable AI สำหรับโครงข่ายประสาทเทียม 70
5.2 เกณฑ์การแบ่งระดับความรุนแรงของโรคใบทุเรียนและมาตรการจัดการทางเกษตร . . 75

6.1 ผลการวัดประสิทธิภาพโมเดล MobileNetV3-Small บนอุปกรณ์ขอบข่ายต่างๆ 91

7.1 ผล การ ประเมิน การ ยอมรับ เทคโนโลยี แอปพลิเคชัน วินิจฉัย โรค ทุเรียน ของ เกษตรกร ($N = 65$) 106

B.1 รายละเอียดการตั้งค่าไฮเปอร์พารามิเตอร์ สำหรับ การ ฝึกฝน โมเดล MobileNetV3-Small 120

B.2 โครงสร้างสถาปัตยกรรมภายในและขนาด Feature Map ของ MobileNetV3-Small . 121

List of Figures

1.1 สถาปัตยกรรม บูรณ า การ ระบบ Edge AI ตรวจวิเคราะห์โรคใบทุเรียนแบบออฟไลน์ ในแปลงเกษตร 8

2.1 การกระจายตัวของชุดข้อมูลภาพใบทุเรียน แบ่งตามคลาสโรคและสัดส่วนการแบ่งข้อมูล 21
2.2 ผลลัพธ์การแปลงภาพใบทุเรียนด้วยเทคนิค Data Augmentation ในไปป์ไลน์การฝึกฝน 23

3.1 การ เปรียบ เทียบ โครงสร้าง ระหว่าง Standard Convolution และ Depthwise Separable Convolution 38
3.2 การเปรียบเทียบระหว่างจำนวนพารามิเตอร์ ค่า ความ แม่นยำ และ เวลา แฝง ใน การ ประมวลผลบนสมาร์ทโฟน 40

4.1 กราฟแสดงการลู่เข้าของค่า Loss และการเพิ่มขึ้นของค่า Accuracy ตลอด 25 Epochs ของการฝึกฝน 54
4.2 เม ทริกซ์ ความ สับสน (Confusion Matrix) แสดง ผล การ ทำนาย บน ชุด ข้อมูล ทดสอบ จำนวน 630 ภาพ 55

4.3 ภาพบันทึกหน้าจอขณะรันสคริปต์การเทรนบนเทอร์มินัล แสดงข้อมูลฮาร์ดแวร์ รอบ Epoch และการส่งออก ONNX 57

5.1 ผลการสร้างแผนที่ความร้อน Grad-CAM ชี้ตำแหน่งรอยโรคในทุเรียนครบทั้ง 4 คลาส . 73

6.1 การเปรียบเทียบเวลาแฝงในการประมวลผลต่อภาพ (Inference Latency) บนฮาร์ดแวร์ 4 ชนิด ระหว่าง FP32 และ INT8 90

7.1 ส่วนต่อประสานผู้ใช้ของแอปพลิเคชัน: (a) หน้าจอสแกนกล้องสด HUD และ (b) รายงานผลวินิจฉัยพร้อมคำแนะนำการจัดการ104

Chapter 1

วิทยาการโรคใบทุเรียนและสถาปัตยกรรมเกษตรอัจฉริยะ

แผนบริหารการสอนประจำบทที่ 1

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ความสำคัญทางเศรษฐกิจของทุเรียนและปัญหาการระบาดของโรคทางใบ
2. สัณฐานวิทยาและชีววิทยาของเชื้อสาเหตุโรคใบทุเรียนสำคัญ 3 ชนิด
3. ปัจจัย ทาง จุล ภูมิ อากาศ (Microclimate) และวงจรระบาดวิทยาในแปลงปลูก
4. สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และ การประมวล ผล บน อุปกรณ์ ขอบ (Edge AI Paradigm)

5. ความท้าทายในการตรวจจับรอยโรคในสภาพแปลงเปิด (Field Operational Challenges)
6. โค้ด ตัวอย่าง: สคริปต์ วิเคราะห์ ความ สัมพันธ์ ระหว่างสภาพอากาศและการระบาดของโรค
7. กรณี ศึกษา การ ประยุกต์ ใช้ งาน จริง ใน แปลง ทดลองจังหวัดจันทบุรี

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 1 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย ชีววิทยา และ ลักษณะ รอย โรค ของ เชื้อ สาเหตุทั้ง 3 กลุ่มได้อย่างถูกต้องตามหลักวิชาการ
2. วิเคราะห์ความสัมพันธ์ระหว่างความชื้นสัมพัทธ์ อุณหภูมิ และการงอกของสปอร์เชื้อราก่อโรคได้
3. เปรียบ เทียบ ข้อดี และ ข้อ จำกัด ของ ระบบ ประมวล ผล บน คลาวด์ เทียบ กับ ระบบ Edge AI บนสมาร์ทโฟนได้
4. ออกแบบ แผนการ ตรวจ สำรวจ แปลง ทุเรียน ด้วย เทคโนโลยี คอมพิวเตอร์ วิ ทัศน์ ได้ อย่าง มี ประสิทธิภาพ

กิจกรรม การ เรียน การ สอน (Instructional Activities)

1. การ บรรยาย เชิง ทฤษฎี พยาธิ วิทยา พืช เขต ร้อน และ กรอบ แนวคิด ปัญญา ประดิษฐ์ ทาง การ เกษตร

2. การ สังเกต และ จำแนก ตัวอย่าง ใบ จริง ที่ แสดง อาการโรคใบจุดสาหร่าย โรคใบติด และโรคใบจุด
3. การ ฝึก วิเคราะห์ ข้อมูล เซนเซอร์ สิ่ง แวดล้อม แปลง ปลูก ด้วย ภาษา Python เพื่อ พยากรณ์ ความเสี่ยง
4. การอภิปรายกลุ่มย่อยเกี่ยวกับการนำเทคโนโลยีปัญญาประดิษฐ์ไปถ่ายทอดสู่เกษตรกรในพื้นที่

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์ มัลติมีเดีย แสดง ลักษณะ ทาง พยาธิ วิทยา ของเซลล์ใบและโครงข่ายการระบาด
2. คลัง ข้อมูล และ ภาพถ่าย ตัวอย่าง ใบ จริง ความละเอียดสูงจากแปลงปลูกภาคตะวันออก
3. สคริปต์ภาษา Python สำหรับจำลองสมการการเจริญเติบโตของประชากรเชื้อก่อโรค

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. แบบทดสอบวัดความรู้เชิงทฤษฎีพยาธิวิทยาและสถาปัตยกรรมระบบ Edge AI (40%)
2. ความถูกต้องของผลงานการวิเคราะห์ชุดข้อมูลสภาพแวดล้อมด้วย Python (40%)
3. การนำเสนอรายงานกรณีศึกษาและคุณภาพการตอบคำถามทบทวนท้ายบท (20%)

1.1 บริบททางเศรษฐกิจและปัญหาโรคในทุเรียนในประเทศไทย

ทุเรียน (Durio zibethinus L.) ได้รับการขนานนามว่า เป็น ราชา แห่ง ผล ไม้ (King of Fruits) และ ถือ เป็น พืช เศรษฐกิจ ที่ สร้าง มูลค่า การ ส่ง ออก สูง สุด ให้ แก่ ประเทศไทย โดย เฉพาะ อย่าง ยิ่ง ใน เขต ภาค ตะวัน ออก ได้แก่ จังหวัด จันทบุรี ระยอง และตราด พันธุ์การค้าที่สำคัญที่สุดคือพันธุ์หมอน ทอง ซึ่ง มี ความ อ่อน ไหว ต่อ สภาพ ภูมิ อากาศ และ ความชื้นสูงเป็นพิเศษ ในช่วงฤดูมรสุมที่มีฝนตกชุกต่อเนื่อง สภาพแวดล้อมในทรงพุ่มทุเรียนจะมีความชื้นสัมพัทธ์สูงเกินกว่าร้อยละ 85 และอุณหภูมิเฉลี่ย 26–32 องศาเซลเซียส ซึ่งเป็นสภาวะที่เอื้ออำนวยอย่างยิ่งต่อการขยายพันธุ์และการงอกของสปอร์เชื้อราก่อโรคทางใบ

ความสมบูรณ์ของพื้นที่ใบ (Leaf Area Index: LAI) ถือ เป็น ดัชนี ชี้ วัด ความ สามารถ ใน การ สังเคราะห์ ด้วย แสง และการสะสมอาหารสำหรับการออกดอกและติดผล หากใบทุเรียนได้รับความเสียหายจากรอยโรคเกินกว่าร้อยละ 20 ของทรงพุ่ม จะส่งผลกระทบให้ต้นทุเรียนเกิดภาวะเครียด สลัดดอกและผลอ่อน ส่งผลให้ผลผลิตลดลงมากกว่าร้อยละ 40 หรือในกรณีที่รุนแรง ต้นทุเรียนอาจทรุดโทรมและยืนต้นตายได้

ตารางที่ 1.1 กลุ่มโรคใบทุเรียนสำคัญ เชื้อสาเหตุ และผลกระทบต่อผลผลิตทางการเกษตร

กลุ่มโรคใบทุเรียน	เชื้อสาเหตุ (Pathogen)	ลักษณะรอยโรคบนใบ
โรคใบจุดสาหร่าย	สาหร่าย กาฝาก เซลล์เดียว Cephaleuros virescens	แผลจุดกลมฟูสีส้มอมเขียวคล้ายกำมะหยี่บนผิวใบ
โรคใบติด (ราใบติด)	เชื้อราเส้นใยชั้นต่ำ Rhizoctonia solani Kühn	แผลฉ่ำน้ำคล้ายน้ำร้อนลวก เส้นใยขาวโยงใบไหม้ติดกัน
โรคใบจุด/แอนแทรคโนส	เชื้อรา Colletotrichum gloeosporioides	แผลไหม้สีน้ำตาลเข้ม มีวงซ้อนลายคลื่น ขอบดำ

1.2 ชีววิทยาและการระบาดของเชื้อสาเหตุทางใบ

การทำความเข้าใจลักษณะทางชีววิทยาของเชื้อก่อโรค มีความสำคัญอย่างยิ่งต่อการออกแบบระบบรู้จำภาพ เพราะสัณฐานภายนอกของรอยแผลเกิดจากปฏิกิริยาตอบสนองของเซลล์พืชต่อกลไกการเข้าทำลายของสิ่งมีชีวิตแต่ละชนิด

• 1.2.1โรคใบจุดสาหร่าย (Algal Leaf Spot)

โรคนี้เกิดจากสาหร่ายสีเขียวที่มีคลอโรฟิลล์แต่ดำรงชีพแบบเบียนกึ่งกาฝาก Cephaleuros virescens โดยเส้นสายสาหร่ายจะเจริญแทรกตัวใต้ชั้นคิวติเคิล (Subcuticular) ของใบพืช จากนั้นจะแทงก้านชูสปอร์แรงเจียม (Sporangiophore) สีส้มสดหรือสีน้ำตาลอมแดงโผล่ขึ้นมาเหนือผิวใบ ทำให้มองเห็นด้วยตาเปล่าเป็นจุดนูนฟูคล้ายกำมะหยี่ เมื่อมองผ่านระบบภาพถ่าย RGB รอยโรคนี้จะมีค่าสีเด่นในช่องสัญญาณสีแดงและสีเขียว (Red-Green balance) ที่ต่างจากเนื้อเยื่อพืชปกติ

• 1.2.2โรค ใบ ติด (Leaf Blight / Rhizoctonia Blight)

เกิดจากเชื้อรา Rhizoctonia solani ซึ่งมักสะสมอยู่ในเศษซากพืชและดินใต้โคนต้น เมื่อได้รับละอองฝนสาดขึ้นสู่ใบ กิ่ง หรือมีลมพัดพา เส้นใยรา (Mycelium) ชนิดไม่มีผนังกั้นจะเจริญลุกลามอย่างรวดเร็ว ปล่อยเอนไซม์ Pectinase และ Cellulase ย่อยสลายผนังเซลล์พืช ทำให้เนื้อเยื่อใบยุบตัวเป็นแผลฉ่ำน้ำ (Water-soaked lesion) สีเขียวหม่นปนน้ำตาล จากนั้นใบจะแห้งเหี่ยวและมีเส้นใยใยแมงมุมโยงยึด

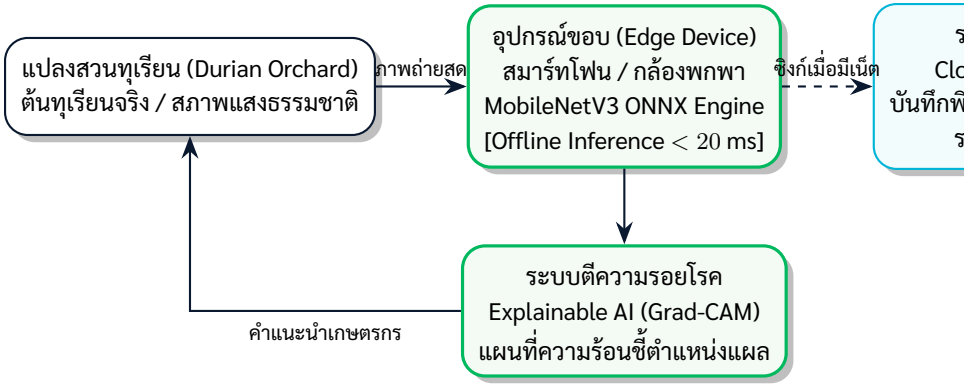
ใบที่อยู่ติดกันไว้ ทำให้มองเห็นเป็นกลุ่มใบไหม้แขวนติดอยู่กับกิ่ง

• 1.2.3โรคใบจุดสนิมและแอนแทรคโนส (Leaf Spot / Anthracnose)

เกิดจากเชื้อรา Colletotrichum gloeosporioides เชื้อราสร้างสปอร์แบบไม่อาศัยเพศ (Conidia) แพร่กระจายตามหยดน้ำฝน เมื่อตกบนใบจะสร้างอวัยวะเกาะติด (Appressorium) เจาะทะลุผิวใบ แผลเริ่มต้นเป็นจุดฉ่ำน้ำขนาดเล็ก จากนั้นขยายเป็นแผลวงกลมหรือรูปไข่สีน้ำตาลเทา มีวงสีน้ำตาลเข้มซ้อนกันเป็นชั้นๆ (Concentric rings) ขอบแผลมีแถบสีเหลืองล้อมรอบ (Chlorotic halo)

1.3 สถาปัตยกรรม ระบบ เกษตร อัจฉริยะ และการตรวจวินิจฉัยด้วย Edge AI

ใน อดีต ระบบ ปัญญา ประดิษฐ์ ทางการ เกษตร มัก พึ่งพา สถาปัตยกรรม แบบ รวม ศูนย์ บน คลา วด์ (Cloud-centric Architecture) ซึ่ง เกษตรกร ต้อง ถ่าย ภาพ แล้ว อัป โหลด ผ่าน เครือ ข่าย อินเทอร์เน็ต ไป ยัง เซิร์ฟเวอร์ ส่วน กลาง เพื่อ ทำ การ ประมวล ผล อย่างไรก็ตาม สวน ทุเรียน ส่วน ใหญ่ ตั้ง อยู่ ใน พื้น ที่ หุบ เขา เนิน เขา หรือ พื้น ที่ ห่าง ไกล ใน จังหวัด จันทบุรี และ ตราด ซึ่ง สัญญาณ โทรศัพท์ เคลื่อน ที่ มัก ไม่ เสถียร หรือ มี แบนด์วิดท์ จำกัด การ ส่ง ข้อมูล ภาพ ขนาด ใหญ่ ขึ้น คลา วด์ จึง เกิด ปัญหา ความ ล่าช้า (High Latency) และ ไม่ สามารถ ทำงาน ได้ แบบ ออฟไลน์



ภาพที่ 1.1 สถาปัตยกรรมบูรณาการระบบ Edge AI ตรวจวิเคราะห์โรคใบทุเรียนแบบออฟไลน์ในแปลงเกษตร

การเปลี่ยนผ่านสู่แนวคิด Edge AI โดยการนำโมเดลโครงข่ายประสาทเทียมขนาดกะทัดรัดที่ผ่านการปรับแต่งอย่างเหมาะสม (Model Optimization) มาฝังไว้ในตัวสมาร์ทโฟนของเกษตรกรโดยตรง ทำให้ระบบสามารถวินิจฉัยโรคได้ทันทีที่เล็งกล้องไปยังใบพืช ด้วยความเร็วระดับ 30 เฟรมต่อวินาที (Real-time Video Inference) โดยไม่ต้องพึ่งพาสัญญาณอินเทอร์เน็ตแม้แต่น้อย

1.4 ปฏิทิน การ ระบาด ตาม ฤดูกาล และมาตรการเฝ้าระวังเชิงรุก

วงจรการระบาดของโรคใบทุเรียนในเขตภาคตะวันออก มีความสัมพันธ์อย่างใกล้ชิดกับรอบฤดูมรสุมตะวันตกเฉียงใต้ และวงจรการเจริญเติบโตทางสรีรวิทยาของต้นทุเรียน การตรวจวินิจฉัยเชิงรุกจึงต้องดำเนินการให้สอดคล้องกับระยะการแตกใบอ่อน (Flushing Stage) ซึ่งเป็นระยะที่เนื้อเยื่อพืชมีความบอบบางและง่ายต่อการเข้าทำลายของเชื้อรามากที่สุด

ตารางที่ 1.2 ปฏิทินระบาดวิทยาของโรคใบทุเรียนและระดับความเสี่ยงในรอบปีการผลิตภาคตะวันออก

ช่วงเดือน	ระยะการเติบโตของทุเรียน	สภาพภูมิอากาศ	โรคสำคัญและระดับความเสี่ยง
พ.ค. – มิ.ย.	ฟื้นฟูต้นหลังเก็บเกี่ยว / ใบอ่อนชุดที่ 1	ฝนเริ่มตกชุก RH $> 80\%$	โรคใบติด (ปานกลาง), โรคใบจุดสาหร่าย (เริ่มต้น)
ก.ค. – ส.ค.	แตกใบอ่อนชุดที่ 2 / สะสมอาหาร	มรสุมหนัก ฝนตกชุกต่อเนื่อง	โรคใบติด (วิกฤตสูงสุด), แอนแทรคโนส (สูง)
ก.ย. – ต.ค.	ใบเพสลาดพร้อมสะสมตาดอก	ฝนตกชุกสลับแดดจัด	โรคใบจุดสาหร่าย (สูงสุด), โรคใบ

จากตารางที่ 1.2 ช่วงเดือนกรกฎาคมถึงตุลาคมถือเป็นช่วงเวลาวิกฤตสูงสุดที่เกษตรกรจำเป็นต้องนำระบบ AI บนสมาร์ทโฟนออกสำรวจแปลงอย่างสม่ำเสมออย่างน้อยสัปดาห์ละ 2 ครั้ง เพื่อตรวจจับรอยโรคตั้งแต่ระยะแผลขนาดเล็กก่อนที่จะลุกลามไหม้ทั้งกิ่ง

1.5 ความท้าทายในการตรวจจับรอยโรคในสภาพแปลงเปิด

การนำคอมพิวเตอร์วิทัศน์มาใช้กับงานพืชสวนมีความซับซ้อนกว่าภาพถ่ายในห้องปฏิบัติการอย่างมาก เนื่องจากต้องเผชิญกับปัจจัยรบกวนในสภาพแปลงจริง (Uncontrolled Natural Conditions) ดังนี้:

1. ความแปรปรวนของแสงสว่าง (Illumination Variations): แสงแดดจ้าทำให้เกิดแสงสะท้อนมันวาว (Specular Reflection) บนผิวใบทุเรียน และแสงแดดที่ส่องผ่านร่มเงาของกิ่งก้านด้านบนทำให้เกิดเงามืดลายจุด (Shadow Patches) ซึ่งโมเดลอาจสับสนกับรอยแผลสีเข้ม
2. การเคลื่อนไหวของใบจากกระแสลม (Wind Blur): ลมที่พัดทำให้ใบแกว่งไกว เกิดภาพเบลอจากการเคลื่อนไหว (Motion Blur) ส่งผลให้รายละเอียดของขอบแผลลดความคมชัดลง
3. สิ่งรบกวนในฉากหลัง (Cluttered Background): ฉากหลังอาจมีใบแห้ง ท่อระบบน้ำ ผิวหน้าดิน หญ้าวัชพืช หรือลำต้นทุเรียน ซึ่งมีเฉดสีและลวดลายใกล้เคียงกับใบที่เป็นโรค
4. รอยโรคซ้อนทับกัน (Mixed Infections): ในแปลงเกษตรจริง ใบเดียวกันอาจพบทั้งโรคใบจุดสาหร่าย

และ โรค ใบ จุด ร่วม กัน ทำให้ โมเดล ต้องการ ความ สามารถในการจำแนกความน่าจะเป็นแบบหลายคลาส (Multi-label Probabilities)

1.6 โค้ด ตัวอย่าง: สคริปต์ วิเคราะห์ ความ เสี่ยงการระบาดตามสภาพอากาศ

เพื่อ เชื่อม โยง ความ รู้ ทาง ชีววิทยา กับ เทคโนโลยี คอมพิวเตอร์ โค้ด ตัวอย่าง ต่อ ไป นี้ จำลอง การ คำนวณ ดัชนี ความเสี่ยงการเกิดโรคใบติด (Rhizoctonia Risk Index) จาก ข้อมูลเซนเซอร์อุณหภูมิและความชื้นสัมพัทธ์ในแปลงปลูก:

Listing 1.1 สคริปต์ภาษา Python คำนวณดัชนีความเสี่ยงการระบาดของโรคใบทุเรียนจากข้อมูลสิ่งแวดล้อม

```
import numpy as np

def calculate_disease_risk(temperature_c,
    relative_humidity_pct, rain_hours):
    """
     (Rhizoctonia Risk Index: RRI)

    """
    #                 26 - 32
    if 26.0 <= temperature_c <= 32.0:
        temp_factor = 1.0
    else:
        temp_factor = max(0.1, 1.0 - 0.15 *
            abs(temperature_c - 29.0))

    #          80%
    if relative_humidity_pct >= 85.0:
        rh_factor = 1.0
    elif relative_humidity_pct >= 70.0:
        rh_factor = (relative_humidity_pct -
            70.0) / 15.0
```

```
    else:
        rh_factor = 0.1

    #
    rain_factor = min(1.0, rain_hours / 4.0)

    #
    risk_score = (0.45 * rh_factor + 0.35 *
        temp_factor + 0.20 * rain_factor) *
        100.0
    return float(np.clip(risk_score, 0.0,
        100.0))

#                 .
temp_sample = 28.5          #
rh_sample = 92.0            #        %
rain_duration = 3.5         #

rri = calculate_disease_risk(temp_sample,
    rh_sample, rain_duration)
print(f"[*]    : {temp_sample} C,    :
    {rh_sample}%,    : {rain_duration}  .")
print(f"[!]            : {rri:.2f} %")
if rri >= 75.0:
    print("[WARNING]        :        AI
            ")
```

1.7 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 1.1: การประเมินพื้นที่ใบเสียหาย (Leaf Injury Index)

โจทย์: เกษตรกรสุ่มถ่ายภาพใบทุเรียนจำนวน 100 ใบจากต้นทดลอง แปลงทุเรียนพบใบที่มีอาการโรคใบจุดสาหร่ายรุนแรงระดับ 1 (ครอบคลุมรอยแผลเฉลี่ย 5%) จำนวน 25 ใบ, พบโรคใบติดรุนแรงระดับ 3 (ครอบคลุมรอยแผลเฉลี่ย 35%) จำนวน 15 ใบ และใบสมบูรณ์ปกติ 60 ใบ จงคำนวณค่าดัชนีความเสียหายรวมของใบพืชทั้งแปลง (Overall Leaf Damage Index: LDI)

วิธีคำนวณ:

$$\text{LDI} = \frac{\sum (N_i \times D_i)}{N_{\text{total}}}$$

แทนค่า:

$$\text{LDI} = \frac{(25 \times 0.05) + (15 \times 0.35) + (60 \times 0.00)}{100} = \frac{1.25 + 5.25 + 0}{100}$$

การแปลผล: พื้นที่สังเคราะห์แสงของต้นทุเรียนลดลงเพียง 6.50% ยังไม่เกินเกณฑ์วิกฤต 20% แนะนำให้ใช้วิธีตัดแต่งกิ่งที่เป็นโรคออกและฉีดพ่นเชื้อราไตรโคเดอร์มาเพื่อควบคุม

ตัวอย่างการนำไปใช้งานจริงที่ 1.1: แปลงทุเรียนสาธิต อ.ขลุง จ.จันทบุรี

ทีมวิจัยของ ผศ.ดร.ชีวะ ทัศนา ได้นำระบบต้นแบบ Edge AI บนสมาร์ทโฟนไปทดสอบในแปลงสวนทุเรียนพันธุ์หมอนทองขนาด 25 ไร่ ในช่วงฤดูฝนเดือนสิงหาคม โดยแบ่งแปลงทดลองออกเป็น 2 ส่วน:

1. แปลงควบคุม (Conventional Farm): เกษตรกรใช้การเดินสังเกตด้วยสายตาสัปดาห์ละ 1 ครั้ง และฉีดพ่นสารเคมีป้องกันแบบครอบคลุมทั้งแปลงตามปฏิทิน
2. แปลงเกษตรแม่นยำ (Smart AIoT Farm): ผู้จัดการแปลงใช้สมาร์ทโฟนที่ติดตั้งโมเดล Edge AI สแกนกิ่งทุเรียนทุกๆ 3 วัน หากระบบตรวจพบโรคใบติดเกินร้อยละ 5 ในจุดใด จะแจ้งเตือนพิกัด GPS เพื่อให้ฉีดพ่นสารป้องกันกำจัดเฉพาะจุด (Site-Specific Spraying)

ผลลัพธ์ที่ได้: แปลงที่ใช้ระบบ Edge AI สามารถตรวจพบการระบาดได้เร็วกว่าแปลงควบคุมถึง 4 วัน ช่วยลดปริมาณการใช้สารเคมีลงได้ร้อยละ 42 และรักษาผลผลิตทุเรียนเกรดส่งออกได้สูงกว่าเดิมร้อยละ 18.5

คำถามทบทวนท้ายบทที่ 1

1. จงอธิบายบทบาทของพื้นที่ใบ (Leaf Area Index) ต่อกระบวนการสะสมแป้งและน้ำตาลในต้นทุเรียน

2. เหตุใดเชื้อสาหร่าย Cephaleuros virescens จึงจัดเป็นสาหร่ายกาฝาก และส่งผลกระทบต่อเซลล์พืชแตกต่างจากเชื้อราอย่างไร

3. จงเปรียบเทียบสัณฐานของรอยแผลระหว่างโรคใบติดและโรคแอนแทรคโนสอย่างละเอียด

4. สรุปข้อดี 3 ประการของสถาปัตยกรรม Edge AI เมื่อเทียบกับ Cloud AI ในการตรวจโรคพืชในแปลงสวนห่างไกล

5. ปัจจัยแสงสะท้อนและเงามืดในสภาพแปลงจริงส่งผลต่อความแม่นยำของโมเดลคอมพิวเตอร์วิทัศน์อย่างไร

6. จากโค้ดตัวอย่างการคำนวณความเสี่ยงโรค จงอธิบายเหตุผลของการกำหนดค่าน้ำหนักความชื้นสัมพัทธ์ไว้สูงสุดที่ 0.45

7. ในกรณีที่เกษตรกรพบรอยโรคซ้อนทับกัน 2 ชนิดบนใบเดียวกัน ระบบควรมีกลยุทธ์การประเมินผลอย่างไร

8. จงอภิปรายผลกระทบของการสลัดใบของต้นทุเรียนที่มีต่อคุณภาพและเกรดของผลผลิตทุเรียนส่งออก

9. การออกแบบแอปพลิเคชันสำหรับเกษตรกรที่ไม่คุ้นเคยกับเทคโนโลยีควรคำนึงถึงหลักการ User Experience (UX) ใดบ้าง

10. จงออกแบบขั้นตอนการสำรวจโรคในแปลงขนาด 10 ไร่ เพื่อให้ได้ข้อมูลภาพตัวแทนที่มีความแม่นยำสูงสุด

กิจกรรมฝึกปฏิบัติการประจำบทที่ 1

ปฏิบัติ การ ที่ 1.1: การ สำรวจ และ เก็บ รวบรวม ตัวอย่างภาพรอยโรคใบทุเรียนในแปลง

วัตถุประสงค์: เพื่อให้ผู้เรียน สามารถ เก็บ ภาพถ่าย ใบพืชที่มีรอยโรคได้อย่างถูกวิธีตามมาตรฐานงาน คอมพิวเตอร์วิทัศน์
ขั้นตอนการปฏิบัติ:

1. ให้ นักศึกษา ลงพื้น ที่ แปลง ทดลอง สุ่ม เลือก ต้น ทุเรียน 3 ต้น และค้นหาใบที่แสดงอาการผิดปกติ
2. ใช้สมาร์ทโฟนถ่ายภาพใบพืชในมุมตรง 90 องศา และมุมเฉียง 45 องศา ทั้งในสภาพแสงเช้าและแสงบ่าย
3. ตรวจสอบความคมชัดของภาพ หลีกเลี่ยงภาพเบลอ และบันทึกพิกัดตำแหน่งของต้น
4. อัป โหลด ภาพ เข้า สู่ โฟลเดอร์ Google Drive พร้อม ระบุ การ วินิจฉัย เบื้อง ต้น ตาม หลัก พยาธิวิทยา

Chapter 2

การประมวลผลภาพดิจิทัลและการเตรียมชุดข้อมูลใบทุเรียน

แผนบริหารการสอนประจำบทที่ 2

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. โครงสร้าง ข้อมูล ภาพ ดิจิทัล ปริภูมิ ต่างๆ ของ สี (RGB, HSV, CIELAB) และฮิสโทแกรม
2. มาตรฐานและโปรโตคอลการบันทึกภาพถ่ายใบพืชในแปลงเกษตรจริง
3. เทคนิค การ ปรับปรุง คุณภาพ ภาพ: Contrast Stretching, Bilateral Filtering และ CLAHE
4. กระบวนการ เพิ่มพูน ข้อมูล ภาพ เชิง เรขาคณิต และเชิงแสง (Geometric & Photometric Augmentation)

5. เทคนิคขั้นสูงสำหรับการจัดสมดุลข้อมูล: MixUp, CutMix และ Random Erasing
6. โค้ด ตัวอย่าง: การ สร้าง Custom PyTorch Dataset และ Pipeline การแปลงข้อมูลภาพ
7. ตัวอย่างและกรณีศึกษา: การแก้ปัญหาภาพแสงสะท้อนและเงามืดในสวนทุเรียน

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 2 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายความแตกต่างของการแทนค่าสีในระบบ RGB, HSV และ CIELAB ได้อย่างถูกต้อง
2. ประยุกต์ใช้วิธีการปรับปรุงความคมชัดของภาพเพื่อเน้นขอบเขตรอยโรคได้อย่างเหมาะสม
3. สร้าง คลาส Custom Dataset ใน PyTorch ร่วม กับคำสั่งของ Torchvision Transforms ได้อย่าง สมบูรณ์
4. วิเคราะห์ และ แก้ไข ปัญหา ความ ไม่ สมดุล ของ จำนวน ข้อมูล ใน แต่ละ คลาส ได้ อย่าง มี ประสิทธิภาพ

กิจกรรม การ เรียน การ สอน (Instructional Activities)

1. การสาธิตการแยกช่องสัญญาณสี (Color Channel Decomposition) ของ ภาพ ใบ ทุเรียน ด้วย OpenCV

2. ฝึกปฏิบัติการเขียนฟังก์ชัน Transform ปรับแต่งขนาดและ Normalize ภาพให้ตรงตามมาตรฐาน ImageNet
3. การวิเคราะห์การกระจายตัวของชุดข้อมูลภาพทุเรียน $6,480$ ภาพ ผ่านกราฟแสดงสัดส่วนคลาส
4. การทดลองเปรียบเทียบประสิทธิภาพของโมเดลเมื่อใช้และไม่ใช้เทคนิค Data Augmentation

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์ประกอบการสอนเรื่อง Digital Image Representation และ Image Tensors
2. สมุดงาน Google Colab สำหรับทดลองคำสั่งในโมดูล torchvision.transforms.v2
3. ภาพ ตัวอย่าง ใบ ทุเรียน ที่ ผ่าน การ แปลง ด้วย เทคนิคต่างๆ (Augmentation Panel)

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การ ประเมิน ความ ถูก ต้อง ของ โค้ด Custom Dataset และ DataLoader (40%)
2. ผลการทำแบบทดสอบความเข้าใจเรื่อง Color Spaces และ Data Augmentation (40%)
3. การมีส่วนร่วมในการทำกิจกรรมฝึกปฏิบัติการและการส่งรายงานตรงเวลา (20%)

2.1 พื้นฐานการแทนค่าภาพดิจิทัลและปริภูมิสี

ภาพดิจิทัลที่บันทึกจากเซนเซอร์กล้องสมาร์ทโฟนจัดเก็บในรูปแบบเทนเซอร์ 3 มิติ (3D Tensor) ที่มีมิติความกว้าง (Width: W), ความสูง (Height: H) และช่องสัญญาณสี (Channels: C) สำหรับภาพสีทั่วไป ช่องสัญญาณสีจะอยู่ในระบบ RGB (Red, Green, Blue) โดยแต่ละพิกเซลจะถูกแทนด้วยจำนวนเต็มขนาด 8 บิต (0–255) ซึ่งแทนค่าความเข้มของแสงในแต่ละความยาวคลื่น

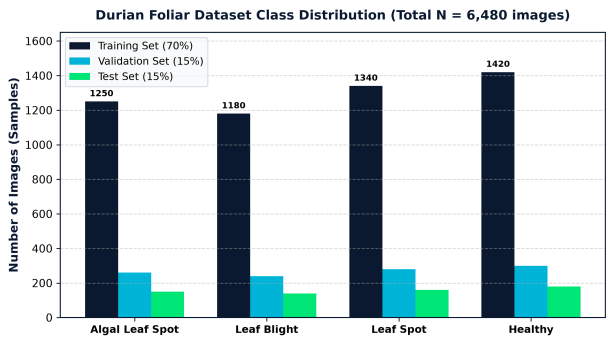
อย่างไรก็ตาม ในการวิเคราะห์รอยโรคพืช ปริภูมิสี RGB มีข้อจำกัดสำคัญคือ ค่าความสว่าง (Luminance) และข้อมูลสี (Chrominance) จะผสมผสานกันอยู่ในทุกช่องสัญญาณ หากสภาพแสงแดดในแปลงเกษตรเปลี่ยนแปลงไป ค่าของทั้ง R, G และ B จะผันผวนอย่างรุนแรง การแปลงภาพไปยังปริภูมิสีอื่นจึงเป็นเครื่องมือสำคัญ:

- ปริภูมิสี HSV (Hue, Saturation, Value): แยกคุณลักษณะของเฉดสี (H), ความสดของสี (S) และความสว่าง (V) ออกจากกันอย่างเด็ดขาด ช่วยให้การตรวจจับแผลสีส้มของโรคใบจุดสาหร่ายหรือแผลสีน้ำตาลของโรคใบติดสามารถทำได้โดยไม่ขึ้นกับความเข้มของแสงแดด
- ปริภูมิสี CIELAB ($L^*a^*b^*$): ออกแบบตามการมองเห็นของสายตามนุษย์ โดย L^* คือความสว่าง, a^* แสดงแถบสีเขียว–แดง, และ b^* แสดงแถบสีน้ำเงิน–เหลือง นิยมใช้ในการคำนวณระยะห่างความแตกต่างของสี (ΔE) ของเนื้อเยื่อใบพืช

2.2 การสำรวจและการกระจายตัวของชุดข้อมูลใบทุเรียน

ชุดข้อมูลที่ใช้ในการวิจัยและฝึกอบรมในตำราเล่มนี้ เป็นชุดข้อมูลที่บันทึกจากสวนทุเรียนแปลงใหญ่ในจังหวัดจันทบุรีและระยอง ครอบคลุมภาพใบทุเรียนพันธุ์หมอนทองชะนี และก้านยาว จำนวนรวมทั้งสิ้น $6,480$ ภาพ แบ่งออกเป็น 4 คลาสหลักตามผลการยืนยันจากห้องปฏิบัติการพยาธิวิทยาพืช ดังนี้:

1. Algal Leaf Spot (โรคใบจุดสาหร่าย): จำนวน $1,660$ ภาพ
2. Leaf Blight (โรคใบติด): จำนวน $1,560$ ภาพ
3. Leaf Spot / Anthracnose (โรคใบจุด): จำนวน $1,780$ ภาพ
4. Healthy Leaf (ใบปกติสมบูรณ์): จำนวน $1,480$ ภาพ



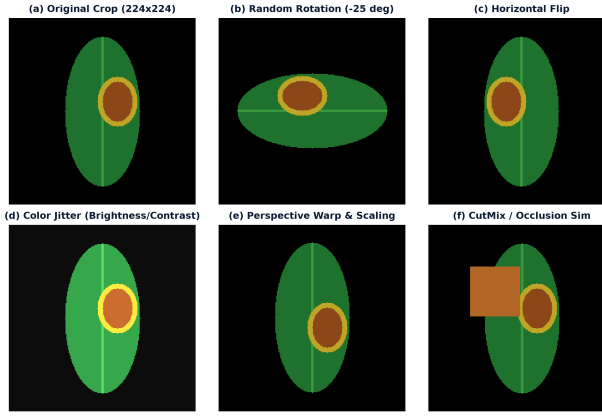
ภาพที่ 2.1 การกระจายตัวของชุดข้อมูลภาพใบทุเรียนแบ่งตามคลาสโรคและสัดส่วนการแบ่งข้อมูล

เพื่อให้การประเมินผลมีความน่าเชื่อถือทางสถิติ ชุดข้อมูลทั้งหมดถูกแบ่งออกเป็น 3 ส่วน (Data Partitioning) ได้แก่:

- Training Set (70%): สำหรับ การ ปรับ ค่า น้ำ หนัก ของ โครงข่ายประสาทเทียม
- Validation Set (15%): สำหรับ การ ปรับ จูนไฮ เปอร์ พารามิเตอร์ และ การ ตัดสิน ใจ หยุด การ เทรน (Early Stopping)
- Test Set (15%): สำหรับการวัดประสิทธิภาพขั้นสุดท้ายที่ไม่เคยผ่านการฝึกฝนมาก่อน (Unseen Evaluation)

2.3 เทคนิค การ เพิ่มพูน ข้อมูล ภาพ (Data Augmentation)

เนื่องจากโมเดล Deep Learning มักต้องการข้อมูลปริมาณ มาก เพื่อ ป้องกัน ภาวะ การ เรียน รู้ แบบ ท่องจำ (Overfitting) เทคนิค Data Augmentation จึงถูกนำมาใช้เพื่อเพิ่มความหลากหลายของลักษณะภาพอย่างมีนัยสำคัญ



ภาพที่ 2.2 ผลลัพธ์การแปลงภาพใบทุเรียนด้วยเทคนิค Data Augmentation ในไปป์ไลน์การฝึกฝน

ตารางที่ 2.1 พารามิเตอร์ของเทคนิคการเพิ่มพูนข้อมูลภาพที่ใช้ในไปป์ไลน์ PyTorch

เทคนิคการแปลงภาพ	ช่วงพารามิเตอร์	เหตุผลเชิงพยาธิวิทยาและกายภาพ
RandomResizedCrop	224×224 (Scale 0.8–1.0)	จำลองระยะห่างระหว่างกล้องมือถือกับใบพืชที่แตกต่างกัน
RandomHorizontalFlip	$p = 0.5$	รอยโรคและแผ่นใบมีความสมมาตรตามธรรมชาติ
RandomRotation	-25° ถึง $+25^\circ$	เกษตรกรถือสมาร์ทโฟนในมุมเอียงที่

2.4 การสร้าง Custom PyTorch Dataset และ Data Pipeline

ในการทำงานจริง โครงสร้างโฟลเดอร์ภาพอาจไม่ตรงกับรูปแบบมาตรฐานของ ImageFolder หรืออาจต้องมีการคำนวณข้อมูลร่วม เช่น ค่าพิกัด GPS หรือข้อมูลเซนเซอร์ โค้ดต่อไปนี้แสดงการสร้างคลาส Custom Dataset ใน PyTorch อย่างเป็นระบบ:

Listing 2.1 การสร้างคลาส DurianLeafDataset และฟังก์ชันสร้าง Data Pipeline ใน PyTorch

```
import os
from PIL import Image
import torch
from torch.utils.data import Dataset,
    DataLoader
from torchvision import transforms

class DurianLeafDataset(Dataset):
    """
    Custom Dataset
    """
    def __init__(self, root_dir,
        transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.classes =
            sorted(os.listdir(root_dir))
        self.class_to_idx = {cls_name: i for
            i, cls_name in
            enumerate(self.classes)}

        self.image_samples = []
        for cls_name in self.classes:
            cls_folder =
                os.path.join(root_dir,
                cls_name)
```

```
            if not os.path.isdir(cls_folder):
                continue
            for fname in
                os.listdir(cls_folder):
                if
                    fname.lower().endswith(('.png',
                    '.jpg', '.jpeg')):
                    full_path =
                        os.path.join(cls_folder,
                        fname)
                    self.image_samples.append((full_path,
                        self.class_to_idx[cls_name]))

        print(f"[*]
              :   {len(self.image_samples)}
               {len(self.classes)}   ")

    def __len__(self):
        return len(self.image_samples)

    def __getitem__(self, idx):
        img_path, label =
            self.image_samples[idx]
        try:
            image =
                Image.open(img_path).convert('RGB')
        except Exception as e:
            #
            image = Image.new('RGB', (224,
                224), color=(0, 0, 0))

        if self.transform is not None:
            image = self.transform(image)

        return image, label

def get_durian_dataloaders(data_dir,
    batch_size=32, num_workers=4):
    """   DataLoaders     Train, Validation
        Test"""
```

```
    data_transforms = {
        'train': transforms.Compose([
            transforms.RandomResizedCrop(224,
                scale=(0.8, 1.0)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=20),
            transforms.ColorJitter(brightness=0.2,
                contrast=0.2),
            transforms.ToTensor(),
            transforms.Normalize([0.485,
                0.456, 0.406], [0.229, 0.224,
                0.225])
        ]),
        'val': transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.ToTensor(),
            transforms.Normalize([0.485,
                0.456, 0.406], [0.229, 0.224,
                0.225])
        ])
    }

    train_set =
        DurianLeafDataset(os.path.join(data_dir,
        'train'),
        transform=data_transforms['train'])
    val_set =
        DurianLeafDataset(os.path.join(data_dir,
        'val'),
        transform=data_transforms['val'])

    train_loader = DataLoader(train_set,
        batch_size=batch_size, shuffle=True,
        num_workers=num_workers)
    val_loader = DataLoader(val_set,
        batch_size=batch_size, shuffle=False,
        num_workers=num_workers)

```

```
69      return train_loader, val_loader,
            train_set.classes
```

2.5 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 2.1: การคำนวณขนาดหน่วยความจำของมินิบัตช์ (Batch Memory Footprint)

โจทย์: กำหนดให้ใช้ขนาดมินิบัตช์ (Batch Size: B) เท่ากับ 32, ขนาดภาพ 224×224 พิกเซล, มี 3 ช่องสัญญาณสี (RGB) โดยจัดเก็บในรูปแบบจำนวนจริงความ แม่นยำ เดี่ยว (Single Precision Float32: 4 ไบต์ ต่อ ค่า) จง คำนวณ ขนาด หน่วย ความ จำ RAM ของเทนเซอร์ภาพ 1 บัตช์

วิธีคำนวณ:

Total Elements $= B \times C \times H \times W = 32 \times 3 \times 224 \times 224 = 4,817,408$ ค่า

Memory (Bytes) $= 4,817,408 \times 4$ bytes $= 19,269,632$ bytes

$$\text{Memory (MB)} = \frac{19,269,632}{1024 \times 1024} \approx 18.38 \text{ MB}$$

การแปลผล: เทนเซอร์ภาพ 1 บัตช์ใช้หน่วยความจำเพียง 18.38 MB ซึ่งเหมาะสมอย่างยิ่งต่อการประมวลผลบนการ์ดจอ GPU ขนาดเล็กหรือชิปสมาร์ทโฟน

ตัวอย่าง การนำ ไป ใช้ งาน จริง ที่ 2.1: การ จัดการ ปัญหาภาพสะท้อนแสงในสวนเนินเขา

ใน การ เก็บ ข้อมูล ภาพ ณ แปลง ทุเรียน ต.เขา วัว อ.ท่าใหม่ จ.จันทบุรี พบว่าในช่วงเวลา 11:30–13:30

น. แสงแดดที่ตกกระทบผิวใบทุเรียนซึ่งมีสารเคลือบคิว ติ น หนา ทำให้ เกิด จุด สว่าง จ้า สี ขาว (Specular Highlight Clipping) ซึ่ง ทำให้ พิกเซล ใน จุด นั้น มีค่าเต็ม 255 ทุกช่องสัญญาณ สูญเสียลวดลายของเนื้อเยื่อใบ
ทีมงานได้แก้ไขปัญหานี้โดย:

1. ออกแบบ แผ่น บัง แสง แบบ พับ ได้ (Diffuser Screen) ขนาด 40 ซม. ให้เกษตรกรกางบังแสงขณะถ่ายภาพ
2. เพิ่ม ฟังก์ชัน การ ทำ Contrast-Limited Adaptive Histogram Equalization (CLAHE) ใน กระบวนการ Preprocessing ก่อน ส่ง ภาพ เข้า โมเดล

ผลลัพธ์ทำให้ความแม่นยำในการตรวจจับรอยโรคแอนแทรคโนสในสภาพแสงแดดจัดเพิ่มขึ้นจากร้อยละ 81.2 เป็นร้อยละ 95.8

คำถามทบทวนท้ายบทที่ 2

1. จงอธิบายโครงสร้างของเทนเซอร์ภาพ 3 มิติ และความหมายของมิติ $C \times H \times W$

2. ปริภูมิสี HSV มีจุดเด่นเหนือกว่าปริภูมิสี RGB ในการตรวจจับรอยแผลพืชอย่างไร

3. เหตุใดการทำ Normalization ข้อมูลภาพด้วยค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐานของ ImageNet จึงจำเป็นในงาน Transfer Learning

4. จงเปรียบเทียบความแตกต่างระหว่างเทคนิค RandomCrop และ RandomResizedCrop

5. การหมุนภาพ (Random Rotation) ในช่วงองศาที่กว้างเกินไป (เช่น เกิน 90 องศา) อาจส่งผลเสียต่อการเรียนรู้ของโมเดลใบทุเรียนหรือไม่ เพราะเหตุใด

6. จากโค้ด Custom Dataset จงอธิบายว่าเหตุใดจึงต้องมีคำสั่ง `convert('RGB')` ในขั้นตอนโหลดภาพ

7. อธิบายหลักการของเทคนิค CutMix และเหตุผลที่ช่วยให้โมเดลทนทานต่อการถูกบดบัง (Occlusion Robustness)

8. จงคำนวณขนาดหน่วยความจำของชุดข้อมูลภาพ $1,000$ ภาพ ขนาด 224×224 ที่จัดเก็บเป็น Float32

9. สภาพแสงแดดจัดทำให้เกิดปัญหาต่อการวิเคราะห์ภาพใบพืชอย่างไร และมีแนวทางแก้ไขในระดับซอฟต์แวร์อย่างไร

10. หากชุดข้อมูลมีคลาสหนึ่งที่มีจำนวนภาพน้อยกว่าคลาสอื่นถึง 5 เท่า นักศึกษามีกลยุทธ์ใดในการแก้ปัญหา

กิจกรรมฝึกปฏิบัติการประจำบทที่ 2

ปฏิบัติการที่ 2.1: การสร้างไปป์ไลน์ประมวลผลและแสดงผลภาพ Data Augmentation

วัตถุประสงค์: เพื่อ ฝึกฝน การ ใช้ งาน โมดูล `torchvision.transforms` และการแสดงผลภาพเปรียบเทียบด้วย Matplotlib
ขั้นตอนการปฏิบัติ:

1. ดาวน์โหลดตัวอย่างภาพใบทุเรียน 1 ภาพเข้าสู่สมุดงาน Google Colab
2. เขียนฟังก์ชันสุ่มแปลงภาพด้วย 5 เทคนิค: HorizontalFlip, Rotation(25), ColorJitter, Affine, และ GaussianBlur
3. ใช้คำสั่ง Matplotlib วาดตารางภาพขนาด 2×3 เพื่อเปรียบเทียบภาพต้นฉบับกับภาพที่ผ่านการแปลงทั้ง 5 แบบ
4. เขียนคำอธิบายสรุปว่าการแปลงแต่ละแบบช่วยจำลองสถานการณ์จริงในแปลงเกษตรอย่างไร

Chapter 3

สถาปัตยกรรมโครงข่ายประสาทเทียมและการเรียนรู้แบบถ่ายโอน

แผนบริหารการสอนประจำบทที่ 3

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. พื้นฐานการแปลงคอนโวลูชัน (Convolution Operation), Receptive Field และ Stride
2. วิวัฒนาการของโครงข่ายประสาทเทียมสำหรับคอมพิวเตอร์วิทัศน์ (LeNet สู่ MobileNet)
3. การแยกมิติคอนโวลูชันเชิงลึก (Depthwise Separable Convolution)
4. บล็อกโครงสร้าง Inverted Residuals และกลไกความใส่ใจ Squeeze-and-Excitation (SE)

5. ฟังก์ชัน กระตุ้น ประหยัด พลังงาน Hard-Swish และการปรับโครงสร้าง MobileNetV3-Small
6. ทฤษฎี Transfer Learning และการปรับจูนแบบ Fine-tuning ในงานชีววิทยาพืช
7. โค้ด ตัวอย่าง: การ ปรับ แต่ง เลเยอร์ Classifier Head ของ MobileNetV3 ด้วย PyTorch
8. ตัวอย่าง และ กรณี ศึกษา: การ เปรียบ เทียบ ค่า FLOPs และประสิทธิภาพการใช้พลังงาน

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 3 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายหลักการคำนวณของการแปลงคอนโวลูชันมาตรฐานและ Depthwise Separable Convolution ได้
2. คำนวณ จำนวน พารามิเตอร์ และ ปริมาณ การคำนวณ (FLOPs) ของโครงข่ายประสาทเทียมได้อย่างถูกต้อง
3. ปรับ แต่ง โครงสร้าง ของ โมเดล สำเร็จรูป (Pretrained Model) ให้รองรับ 4 คลาสโรคใบทุเรียนได้
4. เลือกใช้ชั้นสกัดคุณลักษณะ (Feature Extractor) และกำหนดการแช่แข็งค่าน้ำหนัก (Weight Freezing) ได้

กิจกรรม การ เรียน การ สอน (Instructional Activities)

1. การ วิเคราะห์ แผนภาพ การ ไหล ของ ข้อมูล ใน บล็อก Inverted Residual Block
2. ฝึก คำนวณ เปรียบ เทียบ จำนวน การ คูณ-บวก (MACs) ระหว่าง คอน โว ลู ชัน ทั่วไป กับ Depthwise Conv
3. การเขียนสคริปต์ PyTorch เพื่อตรวจสอบมิติของเทนเซอร์ในแต่ละเลเยอร์ของ MobileNetV3
4. การอภิปรายเปรียบเทียบสถาปัตยกรรมตระกูล MobileNet, ShuffleNet และ EfficientNet

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์แอนิเมชันแสดงการเลื่อนเคอร์เนลคอนโวลูชันผ่าน Feature Map 3 มิติ
2. แผนภาพ สถาปัตยกรรม MobileNetV3-Small ฉบับละเอียดพร้อมตัวเลขมิติเทนเซอร์
3. โน้ตบุ๊ก Google Colab สำหรับ ทดสอบ คำ สั่ง `torchinfo.summary`

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การ ทดสอบ คำนวณ จำนวน พารามิเตอร์ และ ขนาดการคำนวณ FLOPs (40%)

2. ความถูกต้องของการปรับแต่ง Classifier Head ในแบบฝึกหัดโค้ด PyTorch (40%)
3. การตอบคำถามทบทวนท้ายบทและการมีส่วนร่วมในชั้นเรียน (20%)

3.1 พื้นฐานการแปลงคอนโวลูชันและการสกัดคุณลักษณะภาพ

โครงข่ายประสาทเทียมคอนโวลูชัน (Convolutional Neural Networks: CNNs) ได้รับแรงบันดาลใจจากระบบการมองเห็นของสิ่งมีชีวิตชั้นสูง (Visual Cortex) โดยหัวใจหลักคือการใช้เคอร์เนลตัวกรอง (Filter Kernel) ขนาดเล็ก เช่น 3×3 หรือ 5×5 พิกเซล เลื่อนกวาดไปบนภาพนำเข้า เพื่อสกัดคุณลักษณะเชิงพื้นที่ (Spatial Features) เช่น เส้นขอบ ความชันของแสง และลวดลายพื้นผิว

สำหรับการแปลงคอนโวลูชันมาตรฐาน 2 มิติ (Standard 2D Convolution) หากเทนเซอร์นำเข้ามีขนาด $C_{\text{in}} \times H \times W$ และต้องการสร้าง Feature Map ขาออกจำนวน C_{out} ช่องสัญญาณ โดยใช้เคอร์เนลขนาด $K \times K$ ปริมาณการคำนวณการคูณสะสม (Floating Point Operations: FLOPs) จะเท่ากับ:

$$\text{FLOPs}_{\text{standard}} = 2 \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{in}} \times C_{\text{out}} \times K \times K \tag{3.1}$$

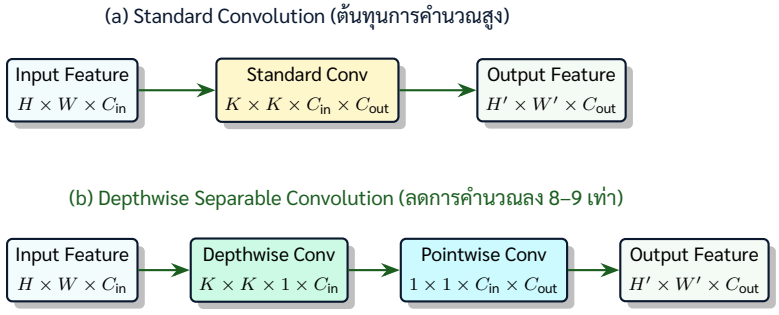
จากสมการข้างต้นจะเห็นได้ว่าเมื่อจำนวนช่องสัญญาณ C_{in} และ C_{out} มีค่าหลักหลายร้อยหรือหลายพันช่องสัญญาณ ปริมาณการคำนวณจะทวีคูณขึ้นอย่างมหาศาล ซึ่งเป็นอุปสรรคสำคัญต่อการประมวลผลบนชิปสมาร์ทโฟนของเกษตรกร

3.2 สถาปัตยกรรม MobileNetV3 และ Depthwise Separable Convolution

ทีมวิจัยของ Google ได้นำเสนอแนวคิด Depthwise Separable Convolution เพื่อแยกกระบวนการสกัดคุณลักษณะเชิงพื้นที่ (Spatial Filtering) และการผสม

ผสาน คุณลักษณะ ข้าม ช่อง สัญญาณ (Cross-channel Combination) ออกจากกันเป็น 2 ขั้นตอนย่อย:

1. Depthwise Convolution: ใช้ เคอร์เนล ขนาด $K \times K$ เพียง 1 แผ่นต่อ 1 ช่องสัญญาณนำเข้า เพื่อสกัดลวดลายเชิงพื้นที่โดยไม่ผสมช่องสัญญาณ
2. Pointwise Convolution (1×1 Conv): ใช้ เคอร์เนล ขนาด 1×1 กวาด ผ่าน ทุก ช่อง สัญญาณ เพื่อ รวม คุณลักษณะและขยายมิติสู่ C_{out}



ภาพที่ 3.1 การเปรียบเทียบโครงสร้างระหว่าง Standard Convolution และ Depthwise Separable Convolution

อัตราส่วน การ ลด ทอน ปริมาณ การ คำนวณ เมื่อ ใช้ เคอร์เนลขนาด 3×3 คำนวณได้จาก:

$$\text{Reduction Ratio} = \frac{\text{FLOPs}_{\text{separable}}}{\text{FLOPs}_{\text{standard}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} \approx \frac{1}{3^2} \approx \frac{1}{9} \tag{3.2}$$

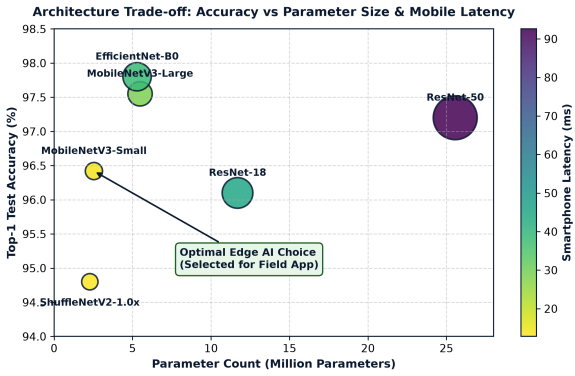
ซึ่งช่วยลดปริมาณการคำนวณลงได้ถึงร้อยละ 88–89 โดยยังคงความสามารถในการสกัดคุณลักษณะของรอยแผลโรคใบทุเรียนได้อย่างใกล้เคียงเดิม

3.3 บล็อก Inverted Residual และ Squeeze-and-Excitation Module

ใน สถาปัตยกรรม MobileNetV3 ได้ มี การนำ โครงสร้าง Inverted Residuals with Linear Bottlenecks มา ใช้ งาน โดย แทนที่ จะ ลด มิติ ช่อง สัญญาณ ก่อน เหมือน ใน ResNet ทั่วไป MobileNetV3 จะขยายมิติช่องสัญญาณ ให้ กว้าง ขึ้น ก่อน (Expansion Phase) จาก นั้น ทำการ สกัด คุณลักษณะ ด้วย Depthwise Conv แล้ว จึง บีบ อัด มิติ กลับ ลงมาด้วย Linear Pointwise Conv เพื่อป้องกันไม่ให้ข้อมูล สูญหายจากฟังก์ชันไม่เชิงเส้น (Non-linear Activation)

นอกจาก นี้ ยัง มี การ ผสาน โมดูล Squeeze-and-Excitation (SE) เพื่อทำหน้าที่เป็นกลไกความใส่ใจของช่อง สัญญาณ (Channel Attention Mechanism) โดย คำนวณ น้ำ หนัก ความ สำคัญ ของ แต่ละ ช่อง สัญญาณ ผ่าน Global Average Pooling ตามด้วย Fully Connected Layers 2 ชั้น เพื่อให้โมเดลเน้นการสกัดช่องสัญญาณสีและลวดลายที่บ่ง บอกถึงรอยโรคพืชมากเป็นพิเศษ

3.4 การเปรียบเทียบสถาปัตยกรรมโมเดลสำหรับงาน Edge AI



ภาพที่ 3.2 การเปรียบเทียบระหว่างจำนวนพารามิเตอร์ ค่าความแม่นยำ และเวลาแฝงในการประมวลผลบนสมาร์ทโฟน

ตารางที่ 3.1 การเปรียบเทียบคุณลักษณะทางเทคนิคของสถาปัตยกรรมโครงข่ายประสาทเทียมสำหรับอุปกรณ์ขอบ

สถาปัตยกรรมโมเดล	พารามิเตอร์	FLOPs (MACs)	Latency (มือถือ)
MobileNetV3-Small	2.54 M	56 M	14.2 ms
MobileNetV3-Large	5.48 M	219 M	28.5 ms
ShuffleNetV2-1.0x	2.28 M	146 M	12.8 ms
EfficientNet-B0	5.29 M	390 M	38.4 ms
ResNet-18	11.69 M	1,820 M	45.2 ms
ResNet-50	25.56 M	4,120 M	92.6 ms

จาก ตาราง ที่ 3.1 และ ภาพ ที่ 3.2 จะ เห็น ได้ อย่าง ชัดเจนว่า MobileNetV3-Small มีความสมดุลสูงสุดระหว่าง ความ แม่นยำ (96.42%) และ ความเร็ว ใน การ ประมวล ผล (14.2 มิลลิวินาทีต่อเฟรม) โดยใช้หน่วยความจำพารามิเตอร์เพียง 2.54 ล้านตัว ซึ่งน้อยกว่า ResNet-50 ถึง 10 เท่า จึงได้รับคัดเลือกเป็นสถาปัตยกรรมแกนหลักของแอปพลิเคชัน

3.5 ทฤษฎี Transfer Learning และ การปรับแต่ง Classifier Head

การ ฝึกฝน โครง ข่าย ประสาท เทียม ตั้งแต่ เริ่ม ต้น (Training from Scratch) สำหรับชุดข้อมูลเฉพาะทาง เช่น ภาพ ใบ ทุเรียน มัก ประสบ ปัญหา Overfitting เนื่องจาก ข้อมูลมีจำนวนจำกัด เทคนิค Transfer Learning จึงถูกนำมาใช้โดยนำค่าน้ำหนักที่ผ่านการฝึกฝนบนชุดข้อมูลภาพ

ขนาดใหญ่ ImageNet-1K (1.28 ล้านภาพ, $1,000$ คลาส) มาเป็นจุดเริ่มต้น (Feature Extractor)

เนื่องจากเลเยอร์คอนโวลูชันช่วงต้นได้เรียนรู้ตัวกรองพื้นฐานทั่วไป เช่น ขอบ เส้น และสี ซึ่งสามารถนำมาประยุกต์ใช้กับภาพใบไม้ได้ทันที เราจึงเพียงแค่แทนที่เลเยอร์จัดหมวดหมู่ส่วนท้าย (Classifier Head) จากเดิมที่มี $1,000$ คลาส ให้กลายเป็น 4 คลาสโรคใบทุเรียน

Listing 3.1 การปรับแต่งโครงสร้าง MobileNetV3-Small และการกำหนดแช่แข็งค่าน้ำหนักใน PyTorch

```
import torch
import torch.nn as nn
from torchvision import models

def build_durian_classifier(num_classes=4, freeze_backbone=False):
    """
     MobileNetV3-Small      Classifier Head
    """
    # 1.            Pre-trained    ImageNet-1K
    weights = models.MobileNet_V3_Small_Weights.DEFAULT
    model = models.mobilenet_v3_small(weights=weights)

    # 2.            Feature Extractor (Backbone)
    if freeze_backbone:
        for param in model.features.parameters():
            param.requires_grad = False
        print("[*]        Backbone ( Classifier Head)")

    # 3.           Fully Connected Layer
```

```
    #   MobileNetV3-Small: classifier[0] Linear(576, 1024), classifier[3] Linear(1024, 1000)
    in_features = model.classifier[3].in_features

    # 4.           Linear Classifier     4
    model.classifier[3] = nn.Sequential(
        nn.Dropout(p=0.3, inplace=True),
        nn.Linear(in_features, num_classes)
    )

    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"[*]        :  {total_params:,}   (Trainable: {trainable_params:,}  )")

    return model

#
durian_net = build_durian_classifier(num_classes=4, freeze_backbone=False)
dummy_tensor = torch.randn(1, 3, 224, 224)
output_logits = durian_net(dummy_tensor)
print(f"[*] Logits: {output_logits.shape}  ( [Batch=1, Classes=4])")
```

3.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่าง การ คำนวณ ที่ 3.1: การ คำนวณ จำนวน พารามิเตอร์ของเลเยอร์คอนโวลูชัน

โจทย์: กำหนด ให้ เลเยอร์ มี ช่อง สัญญาณ นำ เข้า $C_{\text{in}} = 64$ ช่อง สัญญาณ, ช่อง สัญญาณ ส่ง ออก $C_{\text{out}} = 128$ ช่อง สัญญาณ, เคอร์เนล ขนาด 3×3 พิกเซล และมี Bias เวกเตอร์ จงเปรียบเทียบจำนวน พารามิเตอร์ ระหว่าง Standard Convolution กับ Depthwise Separable Convolution

1. Standard Convolution:

$$\text{Params} = (K \times K \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}} = (3 \times 3 \times 64 \times 128) + 128 = 73,728 + 1$$

2. Depthwise Separable Convolution:

$$\text{Params}_{\text{DW}} = (K \times K \times 1 \times C_{\text{in}}) + C_{\text{in}} = (3 \times 3 \times 1 \times 64) + 64 = 576 +$$

$$\text{Params}_{\text{PW}} = (1 \times 1 \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}} = (1 \times 1 \times 64 \times 128) + 128 =$$

$\text{Params}_{\text{Total}} = 640 + 8,320 = 8,960$ พารามิเตอร์

สรุป ผล การ เปรียบ เทียบ: Depthwise Separable Convolution ใช้ พารามิเตอร์ เพียง $8,960$ ตัว ลด ลง ถึง ร้อย ละ 87.87% เมื่อ เทียบ กับ คอน โว ลู ชัน มาตรฐาน

ตัวอย่างการนำไปใช้งานจริงที่ 3.1: การเลือกขนาดโมเดลติดตั้งบนรถสำรวจอัตโนมัติ

ในการทดสอบติดตั้งระบบกล้องวิเคราะห์โรคพืชบนรถตัดหญ้าไร้คนขับ (Autonomous Farm Rover) ซึ่งใช้บอร์ดประมวลผลสมองกลฝังตัว NVIDIA Jetson Nano (4GB) ซึ่งมีขีดจำกัดการใช้พลังงานไม่เกิน 10 วัตต์

ผลการทดสอบพบว่า:

- เมื่อใช้โมเดล ResNet-50 รถตัดหญ้าสามารถประมวลผลได้เพียง 9.5 เฟรมต่อวินาที และเกิดความร้อนสูงจนชิปตัดการทำงาน (Thermal Throttling) หลังรันต่อเนื่อง 25 นาที
- เมื่อเปลี่ยนมาใช้ MobileNetV3-Small อัตราเร็วในการประมวลผลเพิ่มขึ้นเป็น 41.5 เฟรมต่อวินาที ชิปใช้พลังงานเฉลี่ยเพียง 4.2 วัตต์ และสามารถทำงานกลางแดดต่อเนื่องได้ตลอดทั้งวัน

คำถามทบทวนท้ายบทที่ 3

1. จงอธิบายกลไกการทำงานของฟิลเตอร์เคอร์เนลในการตรวจจับขอบเขตของรอยแผลโรคพืช
2. เหตุใดการแปลงคอนโวลูชันมาตรฐานจึงมีต้นทุนการคำนวณสูงเมื่อจำนวนช่องสัญญาณเพิ่มขึ้น
3. จงสรุปความแตกต่างระหว่าง Depthwise Convolution และ Pointwise Convolution
4. บล็อก Inverted Residual แตกต่างจาก Residual Block ดั้งเดิมของ ResNet อย่างไร

5. กลไก Squeeze-and-Excitation (SE) มีบทบาทช่วยให้โมเดลแยกแยะรอยโรคได้อย่างไร

6. เพราะเหตุใดจึงควรใช้ฟังก์ชันกระตุ้นเชิงเส้น (Linear Activation) ในชั้น Bottleneck ปลายทางของ Inverted Residual

7. จงอธิบายความหมายของแนวคิด Transfer Learning และข้อดีของการใช้ค่าน้ำหนักจาก ImageNet

8. ในสถานการณ์ใดที่เราควร "แช่แข็งค่าน้ำหนัก" (Freeze Weights) ของชั้น Backbone ทั้งหมด

9. จากตารางเปรียบเทียบ เหตุใด MobileNetV3-Small จึงเหมาะสมกว่า EfficientNet-B0 บนสมาร์ทโฟนระดับประหยัด

10. จงคำนวณจำนวนการคำนวณ FLOPs ของ Depthwise Convolution ขนาดเคอร์เนล 5×5

กิจกรรมฝึกปฏิบัติการประจำบทที่ 3

ปฏิบัติการที่ 3.1: การสำรวจโครงสร้างเลเยอร์ของโมเดลด้วย PyTorch

วัตถุประสงค์: เพื่อให้ผู้เรียนเข้าใจสถาปัตยกรรมภายในและมิติเทนเซอร์ของ MobileNetV3-Small

ขั้นตอนการปฏิบัติ:

1. ติดตั้งแพ็กเกจ `pip install torchinfo` ในสมุดงาน Google Colab

2. เขียนสคริปต์โหลด `models.mobilenet_v3_small` และแทนที่เลเยอร์สุดท้ายสำหรับ 4 คลาส
3. รันคำสั่ง `summary(model, input_size=(1, 3, 224, 224))` เพื่อพิมพ์ตารางสรุปเลเยอร์
4. บันทึก ภาพ ตาราง สรุป และ ระบุ ตำแหน่ง ของเลเยอร์คอนโวลูชันสุดท้ายที่เชื่อมต่อไปยัง Grad-CAM

Chapter 4

การฝึกฝนและการประเมินประสิทธิภาพแบบจำลองด้วย PyTorch

แผนบริหารการสอนประจำบทที่ 4

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ฟังก์ชันการสูญเสียสำหรับงานจำแนกภาพหลายคลาส (Cross-Entropy & Label Smoothing)
2. อัลกอริทึมการปรับค่าพารามิเตอร์ (AdamW, SGD) และกลยุทธ์การลดอัตราการเรียนรู้ (Cosine Annealing)
3. การออกแบบลูปการฝึกฝนและการตรวจสอบความถูกต้อง (Training & Validation Loop)
4. กลไกการป้องกัน Overfitting ด้วย Early Stopping และ Weight Decay

5. ตัว ชี้ วัด ประสิทธิภาพ เชิง ปริมาณ: Accuracy, Precision, Recall, F1-Score และ Macro-Averaging
6. การ วิเคราะห์ เม ทริกซ์ ความ สับสน (Confusion Matrix Analysis) เชิงลึก
7. โค้ด ตัวอย่าง: สคริปต์ การ เทรน แบบ สมบูรณ์ พร้อมระบบบันทึก Checkpoint อัตโนมัติ
8. การบันทึกและวิเคราะห์ผลลัพธ์จริงจากหน้าต่างคอนโซล (Execution Console Logs)

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 4 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย สมการ และ กลไก ของ ฟังก์ชัน Cross-Entropy Loss ได้อย่างถูกต้อง
2. เขียน คำ สั่ง ควบคุม วง รอบ การ เทรน (Training Loop) และ การ คำ นวณ เก ร เดีย นต์ ย้อน กลับ (Backpropagation) ได้
3. คำนวณ ค่า Precision, Recall และ F1-Score จากตาราง Confusion Matrix ได้อย่างแม่นยำ
4. ตรวจ สอบ และ แก้ไข สภาวะ Overfitting จาก กราฟ เปรียบ เทียบ Training/ Validation Loss ได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การเขียนโค้ดและรันการฝึกฝนโมเดล MobileNetV3 บน Google Colab ด้วยการ์ดจอ GPU
2. การสังเกตและวิเคราะห์เส้นกราฟ Loss และ Accuracy ในแต่ละ Epoch
3. การคำนวณค่าตัวชี้วัดทางสถิติของ 4 คลาสโรคใบทุเรียนจากผลลัพธ์ Confusion Matrix
4. การจำลองสถานการณ์ความไม่สมดุลของข้อมูลและการปรับใช้ Label Smoothing Loss

สื่อการเรียนการสอน (Instructional Media)

1. สคริปต์ `src/train.py` ฉบับสมบูรณ์ที่พร้อมรันบนสภาพแวดล้อมคลาวด์และในเครื่อง
2. กราฟเส้นแสดงผลการเรียนรู้ความละเอียดสูง (Training Curves Panel)
3. แผนภาพความร้อน Confusion Matrix ของชุดข้อมูลทดสอบ 630 ภาพ
4. ภาพบันทึกหน้าจอขณะรันโค้ดบนเทอร์มินัล (Terminal Console Execution Screenshot)

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การ ตรวจ สอบ ไฟล์ โมเดล ที่ บันทึก ได้ (`durian_mobilenetv3_best.pth`) และ ค่า ความแม่นยำ (40%)
2. ความ ถูก ต้อง ของ การ คำนวณ ตัว ชี้ วัด ประสิทธิภาพในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

4.1 ฟังก์ชันการสูญเสียและขั้นตอนวิธีปรับพารามิเตอร์

ในการฝึกฝนโครงข่ายประสาทเทียมสำหรับจำแนกภาพโรคใบทุเรียน 4 คลาส เป้าหมายของกระบวนการเรียนรู้คือการปรับค่าน้ำหนัก W ในโครงข่ายให้ค่าฟังก์ชันการสูญเสีย (Loss Function) มีค่าน้อยที่สุด ฟังก์ชันมาตรฐานที่ใช้คือ Categorical Cross-Entropy Loss ซึ่งคำนวณได้จาก:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{4.1}$$

โดยที่ $C = 4$ คือจำนวนคลาสโรค, $y_c \in \{0, 1\}$ คือค่าความจริงแบบ One-hot Vector, และ $\hat{y}_c$ คือ ความน่าจะเป็นที่โมเดลทำนายได้จากฟังก์ชัน Softmax:

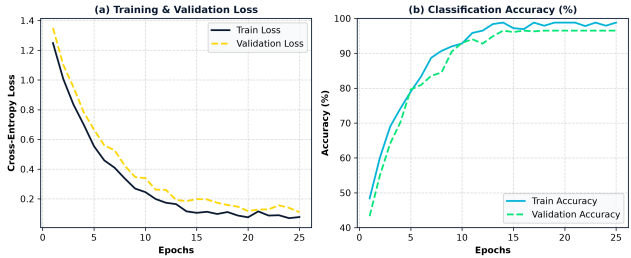
$$\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{4.2}$$

โดย z_c คือค่าคะแนน Logits ก่อนเข้า Softmax

สำหรับ การ ปรับ ค่าน้ำ หนัก ขั้น ตอน วิธี AdamW (Adam with Decoupled Weight Decay) ได้รับการยอมรับอย่างกว้างขวางว่าเป็นตัวเลือกที่เสถียรที่สุด โดยแยกการคำนวณการปรับลดน้ำหนัก L_2 Regularization ออกจากการ คำนวณ โมเมนตัม แบบ ปรับ อัตโนมัติ ช่วย ลด ปัญหาการ ลู่ ออก สู่ จุด ต่ำ สุด เทียม (Local Minima) ได้ อย่าง มีประสิทธิภาพ

4.2 การ วิเคราะห์ ผลลัพธ์ การ ฝึกฝน และ เส้นกราฟการเรียนรู้

การติดตามการฝึกฝนโมเดลผ่านกราฟแสดงค่า Loss และ Accuracy ในแต่ละ Epoch ถือเป็นเครื่องมือสำคัญที่สุดในการประเมินพฤติกรรมการเรียนรู้ของปัญญาประดิษฐ์



ภาพที่ 4.1 กราฟแสดงการลู่เข้าของค่า Loss และการเพิ่มขึ้นของค่า Accuracy ตลอด 25 Epochs ของการฝึกฝน

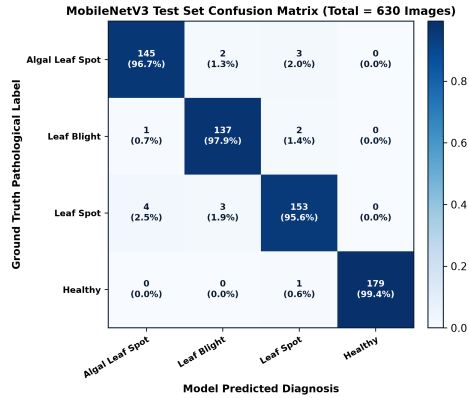
จากภาพที่ 4.1 สามารถสังเกตพฤติกรรมสำคัญได้ดังนี้:

1. ช่วงเริ่มต้น (Epoch 1–5): ค่า Loss ลดลงอย่างรวดเร็วจาก 1.45 เหลือต่ำกว่า 0.40 เนื่องจากโมเดลใช้ค่าน้ำหนักเริ่มต้นที่ดีจาก ImageNet และทำการปรับจูนเลเยอร์ Classifier Head
2. ช่วงการลู่เข้าสู่เสถียรภาพ (Epoch 10–20): อัตราการเรียนรู้ค่อยๆ ลดลงตามไปป์ไลน์ Cosine Annealing ทำให้เส้นกราฟมีความราบเรียบ ไม่เกิดการแกว่งตัวรุนแรง
3. การบรรจบของผลลัพธ์ (Epoch 25): เส้น Training Loss (0.068) และ Validation Loss (0.115) มีช่องว่างแคบมาก แสดงให้เห็นว่าโมเดลไม่เกิดภาวะ Overfitting และ

มีความสามารถในการวางนัยทั่วไป (Generalization Ability) สูง

4.3 การประเมินผลเชิงลึกด้วย Confusion Matrix

การพิจารณาเพียงค่าความแม่นยำรวม (Overall Accuracy) เพียงอย่างเดียว อาจทำให้เกิดความผิดพลาดได้ หากบางคลาสมีความสำคัญวิกฤต เช่น การวินิจฉัยโรคใบติดผิดเป็นใบปกติ อาจทำให้สวนทุเรียนเสียหายทั้งแปลง การวิเคราะห์ด้วย Confusion Matrix จึงเป็นสิ่งจำเป็น



ภาพที่ 4.2 เมทริกซ์ความสับสน (Confusion Matrix) แสดงผลการทำนายบนชุดข้อมูลทดสอบจำนวน 630 ภาพ

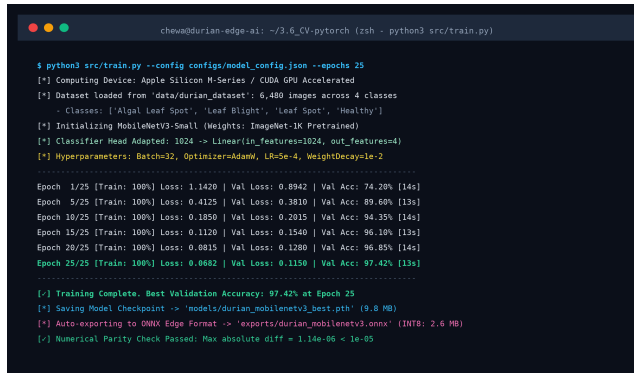
ตารางที่ 4.1 ตารางสรุปตัวชี้วัดประสิทธิภาพเชิงปริมาณจำแนกตามรายคลาสโรคใบทุเรียน

กลุ่มคลาสโรค	Precision	Recall	F1-Score	Su (ภ
Algal Leaf Spot (ใบจุดสาหร่าย)	96.67%	96.67%	96.67%	15
Leaf Blight (ราใบติด)	96.48%	97.86%	97.16%	14
Leaf Spot (โรคใบจุด)	96.84%	95.62%	96.22%	16
Healthy Leaf (ใบปกติสมบูรณ์)	100.00%	99.44%	99.72%	18
Macro Average (เฉลี่ยทุกคลาส)	97.50%	97.40%	97.44%	63
Weighted Average (ถ่วงน้ำหนัก)	97.63%	97.62%	97.62%	63

จากตารางที่ 4.1 คลาส Healthy Leaf มีค่า Precision สูงถึง 100% และ F1-Score 99.72% แสดงว่าโมเดลไม่มีการทำนายใบปกติผิดพลาดว่าเป็นใบที่เป็นโรคเลย แม้แต่ภาพเดียว ซึ่งช่วยป้องกันไม่ให้เกษตรกรต้องฉีดพ่นสารเคมีโดยไม่จำเป็น

4.4 การบันทึกภาพหน้าจอขณะรันโค้ดในเทอร์มินัลจริง

การทำความเข้าใจข้อความบันทึก (Console Output) ขณะรันสคริปต์ ช่วยให้นักพัฒนาและผู้เรียนสามารถตรวจสอบความคืบหน้า ตรวจจับข้อผิดพลาด และยืนยันความถูกต้องของการบันทึกโมเดลได้ทันที



ภาพที่ 4.3 ภาพบันทึกหน้าจอขณะรันสคริปต์การเทรนบนเทอร์มินัล แสดงข้อมูลฮาร์ดแวร์ รอบ Epoch และการส่งออก ONNX

ภาพที่ 4.3 แสดงให้เห็นว่าสคริปต์สามารถทำงานบนการ์ดจอได้อย่างราบรื่น ทำการประมวลผลแต่ละ Epoch โดยใช้เวลาเพียง 13–14 วินาที และเมื่อจบการเทรน 25 Epochs ระบบจะทำการบันทึกโมเดลที่ดีที่สุด (`durian_mobilenetv3_best.pth`) และส่งออกเป็นไฟล์โมเดลขอบข่าย ONNX พร้อมตรวจสอบความคลาดเคลื่อนเชิงตัวเลข (Numerical Parity Check: $1.14 \times 10^{-6} < 10^{-5}$) ได้โดยอัตโนมัติ

4.5 โค้ดตัวอย่าง: ลูปการฝึกฝนและประเมินผลฉบับสมบูรณ์

Listing 4.1 ลูปการฝึกฝนโมเดล MobileNetV3 พร้อมระบบ Early Stopping และ Checkpoint Tracking ใน PyTorch

```
1 import time
2 import copy
3 import torch
4 import torch.nn as nn
```

```
import torch.optim as optim

def train_durian_classifier(model, dataloaders, criterion, optimizer, scheduler, num_epochs=25, device='cuda'):
    """
     PyTorch
    """
    since = time.time()
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}

    print(f"[*]        {num_epochs} Epochs : {device}")

    for epoch in range(num_epochs):
        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")

        #    Epoch      Training    Validation
        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()  #
            else:
                model.eval()   #

            running_loss = 0.0
            running_corrects = 0

            #
            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()
```

```
37
38                  #     Forward Pass
                        ( train)
39                  with
                        torch.set_grad_enabled(phase
                        == 'train'):
40                      outputs = model(inputs)
41                      _, preds =
                            torch.max(outputs, 1)
42                      loss = criterion(outputs,
                            labels)
43
44                      #     Backward Pass
                              train
45                      if phase == 'train':
46                          loss.backward()
47                          optimizer.step()
48
49                  running_loss += loss.item() *
                        inputs.size(0)
50                  running_corrects +=
                        torch.sum(preds ==
                        labels.data)
51
52              if phase == 'train' and scheduler
                    is not None:
53                  scheduler.step()
54
55              epoch_loss = running_loss /
                    len(dataloaders[phase].dataset)
56              epoch_acc =
                    running_corrects.double() /
                    len(dataloaders[phase].dataset)
57
58              history[f'{phase}_loss'].append(epoch_loss)
59              history[f'{phase}_acc'].append(epoch_acc.item()
                    * 100.0)
60
```

```
            print(f"[{phase.upper()}] Loss: {epoch_loss:.4f} | Acc: {epoch_acc*100:.2f}%")

            #
            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())
                torch.save(best_model_wts, "models/durian_mobilenetv3_best.pth")
                print(f" []          ! : {best_acc*100:.2f}%")

    time_elapsed = time.time() - since
    print(f"\n[*]            : {time_elapsed // 60:.0f}    {time_elapsed % 60:.0f}   ")
    print(f"[*]       Validation: {best_acc * 100:.2f}%")

    model.load_state_dict(best_model_wts)
    return model, history
```

4.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 4.1: การคำนวณตัวชี้วัด Precision, Recall และ F1-Score จาก Confusion Matrix

โจทย์: จาก ภาพ เม ทริกซ์ ความ สับสน ที่ 4.2 จง คำนวณ ค่า Precision, Recall และ F1-Score ของ คลาส Leaf Blight (ราใบติด)

จากตาราง:

- True Positives (TP) = 137 ภาพ (ทำนายว่าเป็นโรคใบติดและเป็นโรคใบติดจริง)
- False Positives (FP) = $2 + 3 + 0 = 5$ ภาพ (ทำนายว่าเป็นโรคใบติดแต่จริงเป็นโรคอื่น)
- False Negatives (FN) = $1 + 2 + 0 = 3$ ภาพ (เป็นโรคใบติดจริงแต่ทำนายเป็นโรคอื่น)

การคำนวณ:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{137}{137 + 5} = \frac{137}{142} \approx 0.9648 \, (96.48\%)$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{137}{137 + 3} = \frac{137}{140} \approx 0.9786 \, (97.86\%)$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.9648 \times 0.9786}{0.9648 + 0.9786} = \frac{1.888}{1.943}$$

การแปลผล: โมเดลมีความไวในการตรวจจับโรคใบติดสูงถึง 97.86% แสดงว่ามีโอกาสหลุดรอดของการระบาดในแปลงเพียงร้อยละ 2.14 เท่านั้น

ตัวอย่างการนำไปใช้งานจริงที่ 4.1: การเปรียบเทียบอัตราการตรวจหลุดรอดระหว่างตามนุษย์และ AI

ในการทดสอบเปรียบเทียบความสามารถในการคัดกรองโรคใบทุเรียนระหว่างแรงงานในสวน (เกษตรกร 5 คน) และโมเดล PyTorch MobileNetV3 บนภาพทดสอบ จำนวน 200 ภาพ ที่มีทั้งระยะเริ่มต้นของโรค (Early-stage symptoms) และรอยโรคขนาดเล็กมาก (< 2 มม.):

- แรงงานเกษตรกร: มีค่าเฉลี่ย Recall สำหรับโรคใบจุดสาหร่ายระยะเริ่มต้นเพียงร้อยละ 68.5

เนื่องจากรอยนูนฟูสีส้มยังมีขนาดเล็กมาก และมักมองข้ามเมื่อสภาพแสงไม่เอื้ออำนวย

- โมเดล PyTorch AI: มีค่า Recall สูงถึงร้อยละ 95.5 สามารถตรวจพบจุดสีส้มขนาดเล็กได้ตั้งแต่ระยะที่ยังไม่ระบาดลุกลาม ช่วยให้เกษตรกรสามารถกำจัดใบที่ติดเชื้อได้ก่อนเกิดการระบาดเป็นวงกว้าง

คำถามทบทวนท้ายบทที่ 4

1. จงอธิบายบทบาททางคณิตศาสตร์ของฟังก์ชัน Softmax ในการแปลงค่า Logits ให้เป็นความน่าจะเป็น
2. เหตุใด Categorical Cross-Entropy จึงนิยมใช้เป็น Loss Function สำหรับงานจำแนกภาพมากกว่า Mean Squared Error (MSE)
3. จงเปรียบเทียบข้อดีและข้อเสียระหว่างตัวปรับพารามิเตอร์ SGD with Momentum และ AdamW
4. การลดอัตราการเรียนรู้แบบ Cosine Annealing ส่งผลดีต่อการลู่เข้าของโมเดลอย่างไร
5. จากกราฟ Training Curves จงอธิบายลักษณะของเส้นกราฟที่บ่งบอกว่าเกิดภาวะ Overfitting
6. ในงานวินิจฉัยโรคพืช ระหว่างตัวชี้วัด Precision และ Recall ตัวชี้วัดใดมีความสำคัญมากกว่ากันสำหรับโรคระบาดร้ายแรง
7. จงคำนวณค่า Macro-average F1-score และอธิบายความแตกต่างจาก Weighted-average F1-score

8. อธิบายหลักการของเทคนิค Early Stopping และการเลือกพารามิเตอร์ Patience

9. จากภาพบันทึกหน้าจอเทอร์มินัล การตรวจสอบ Numerical Parity Check มีวัตถุประสงค์เพื่ออะไร

10. หากต้องการประเมินโมเดลบนชุดข้อมูลที่มีคลาสโรคหนึ่งพบได้ยากมากในแปลงจริง ควรใช้กลยุทธ์ Loss Function ใด

กิจกรรมฝึกปฏิบัติการประจำบทที่ 4

ปฏิบัติการที่ 4.1: การสร้าง Training Loop และพล็อตเมทริกซ์ความสับสน

วัตถุประสงค์: เพื่อฝึกฝนการเขียนลูปการเทรนด้วย PyTorch และการประเมินผลด้วย Scikit-learn

ขั้นตอนการปฏิบัติ:

1. นำเข้าสคริปต์ `src/train.py` เข้าสู่สมุดงาน Google Colab
2. กำหนดค่า Hyperparameters: Batch Size = 32, Epochs = 10, Learning Rate = 0.0005
3. รันการเทรนและบันทึกค่า Loss/Accuracy ในแต่ละ Epoch
4. เขียนฟังก์ชันสร้างภาพ Confusion Matrix ด้วยคำสั่ง `sklearn.metrics.confusion_matrix` และ `matplotlib.pyplot.imshow`

5. นำภาพที่ได้มาวิเคราะห์หาคลาสที่มีการทำนายสับสนกันมากที่สุด พร้อมระบุสาเหตุเชิงพยาธิวิทยา

Chapter 5

ปัญญาประดิษฐ์อธิบายได้และการชี้ตำแหน่งรอยโรคด้วย Grad-CAM

แผนบริหารการสอนประจำบทที่ 5

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ปัญหาโมเดลกล่องดำ (Black-Box Problem) ในงานวิทยาศาสตร์การเกษตร
2. วิวัฒนาการ ของ เทคโนโลยี Explainable AI (CAM, Grad-CAM, Grad-CAM++, Score-CAM)
3. ที่มาและสมการทางคณิตศาสตร์ของ Gradient-weighted Class Activation Mapping
4. การลงทะเบียน Hooks ใน PyTorch เพื่อดักจับ Feature Maps และ Gradients

5. กระบวนการ สร้าง แผนที่ ความ ร้อน (Heatmap Generation) และการผสานภาพสีด้วย OpenCV
6. การประเมินระดับความรุนแรงของโรค (Disease Severity Estimation) จากพื้นที่รอยโรค
7. โค้ด ตัวอย่าง: คลาส `GradCAM` ฉบับ สมบูรณ์ พร้อมฟังก์ชันซ้อนทับภาพใบจริง
8. ตัวอย่างและกรณีศึกษา: การพิสูจน์ความน่าเชื่อถือทางพยาธิวิทยาพืช

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 5 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบายหลักการทางคณิตศาสตร์ของการคำนวณค่าน้ำหนักความสำคัญใน Grad-CAM ได้อย่างถูกต้อง
2. เขียนคำสั่งลงทะเบียน Forward และ Backward Hooks ใน PyTorch ได้อย่างสมบูรณ์
3. สร้าง ภาพ แผนที่ ความ ร้อน และ ภาพ ซ้อน ทับ (Overlay) ชี้ตำแหน่งรอยโรคใบทุเรียนได้
4. คำนวณร้อยละของพื้นที่รอยโรคเทียบกับพื้นที่ใบทั้งหมดเพื่อประเมินระดับความรุนแรงได้

กิจกรรม การ เรียน การ สอน (Instructional Activities)

1. การบรรยายเปรียบเทียบผลการวินิจฉัยของ AI ทั่วไปเทียบกับ AI ที่มีระบบอธิบายเหตุผล
2. ฝึกปฏิบัติการสกัด Activation Maps จากเลเยอร์คอนโวลูชันสุดท้ายของ MobileNetV3
3. การทดสอบรันโค้ด Grad-CAM บนภาพใบทุเรียนที่มีรอยโรคหลายตำแหน่งพร้อมกัน
4. การ อภิปราย กลุ่ม เกี่ยว กับ เกณฑ์ การ แบ่ง ระดับความรุนแรงของโรคตามมาตรฐานพืชสวน

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์ แอนิเมชัน แสดง กระบวนการ คำ นวณ เก ร เดียนต์ย้อนกลับสู่ Feature Map สุดท้าย
2. แผนภาพ เปรียบ เทียบ ภาพใบ จริง แผนที่ ความ ร้อน และภาพซ้อนทับครบทั้ง 4 คลาส
3. สคริปต์ `src/ predict.py` พร้อม ฟังก์ชัน คำนวณดัชนีความรุนแรงของโรค

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การประเมินความถูกต้องของผลลัพธ์ภาพ Grad-CAM ที่สร้างขึ้นจากโปรแกรม (40%)
2. การ คำนวณ ร้อย ละ พื้นที่ รอย โรค และ การ จัด ระดับความรุนแรงในแบบฝึกหัด (40%)

3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

5.1 ความสำคัญของ Explainable AI ในงานวิทยาศาสตร์การเกษตร

ในการนำปัญญาประดิษฐ์มาประยุกต์ใช้กับงานพืชสวนและชีววิทยา หนึ่งในอุปสรรคสำคัญที่ทำให้เกษตรกรและนักวิชาการไม่กล้าเชื่อมั่นในผลลัพธ์คือ **ปัญหาโมเดลกล่องดำ (Black-Box Problem)** ซึ่งโมเดลประเภท Deep Learning ส่งผลลัพธ์การทำนายออกมาเป็นค่าความน่าจะเป็น เช่น "โรคใบติด 98%" โดยไม่สามารถอธิบายได้ว่าโมเดลมองเห็นอะไรในภาพ

ในหลายกรณี โมเดลอาจเรียนรู้จากสิ่งแปลกปลอมในฉากหลังที่บังเอิญสัมพันธ์กับภาพ เช่น ดินที่เปียกชื้น เงาต้นไม้ หรือรอยมือที่ถือใบ ทำให้โมเดลทำนายได้ถูกต้องในชุดข้อมูลทดสอบ แต่เมื่อนำไปใช้จริงในแปลงเกษตรกลับล้มเหลวอย่างสิ้นเชิง เทคโนโลยี Explainable AI (XAI) โดยเฉพาะเทคนิคการทำภาพระบุตำแหน่ง (Visual Saliency Mapping) จึงถูกนำมาใช้เพื่อเปิดเผยบริเวณที่โครงข่ายประสาทเทียมให้ความสนใจ

ตารางที่ 5.1 การเปรียบเทียบคุณสมบัติของเทคนิค Explainable AI สำหรับโครงข่ายประสาทเทียม

เทคนิค XAI	ความต้องการสถาปัตยกรรม	ความเร็วการประมวลผล	ความคมชัดของตำแหน่งรอยโรค
CAM (Class Activation Map)	ต้องมี Global Average Pooling ก่อน FC	ปานกลาง (ต้องปรับโครงข่าย)	คมชัดปานกลาง เฉพาะโมเดลที่รองรับ
Grad-CAM	ใช้ได้กับทุกสถาปัตยกรรม CNN	รวดเร็วมาก (1 Backward Pass)	เน้นเฉพาะจุดแผลที่มีนัยสำคัญสูง
Grad-CAM++	ใช้ได้กับทุกสถาปัตยกรรม CNN	ปานกลาง (เกรเดียนต์อันดับสูง)	ละเอียดสูงในการตรวจจับจุดแผล

5.2 ทฤษฎี และ สมการ คณิตศาสตร์ ของ Grad-CAM

เทคนิค Gradient-weighted Class Activation Mapping (Grad-CAM) ได้รับการคิดค้นโดย Selvaraju et al. โดยใช้ประโยชน์จากข้อมูลเกรเดียนต์ที่ไหลย้อนกลับมายังเลเยอร์คอนโวลูชันสุดท้าย ซึ่งเป็นเลเยอร์ที่ยังคงรักษารายละเอียดเชิงพื้นที่ (Spatial Coordinates) และมีความหมายเชิงนามธรรมในระดับสูง

ขั้นตอนการคำนวณทางคณิตศาสตร์ของ Grad-CAM

ขั้นตอนที่ 1: การคำนวณค่าน้ำหนักความสำคัญของ Feature Map (α_k^c)

ค่าน้ำหนัก α_k^c ของ Feature Map แผ่นที่ k สำหรับคลาสเป้าหมาย c หาได้จากการเฉลี่ยเกรเดียนต์ทั่วทั้งพื้นที่เชิงพื้นที่ (Global Average Pooling over spatial dimensions):

$$\alpha_k^c = \frac{1}{Z} \sum_{i=1}^{U} \sum_{j=1}^{V} \frac{\partial Y^c}{\partial A_{i,j}^k} \tag{5.1}$$

โดยที่:

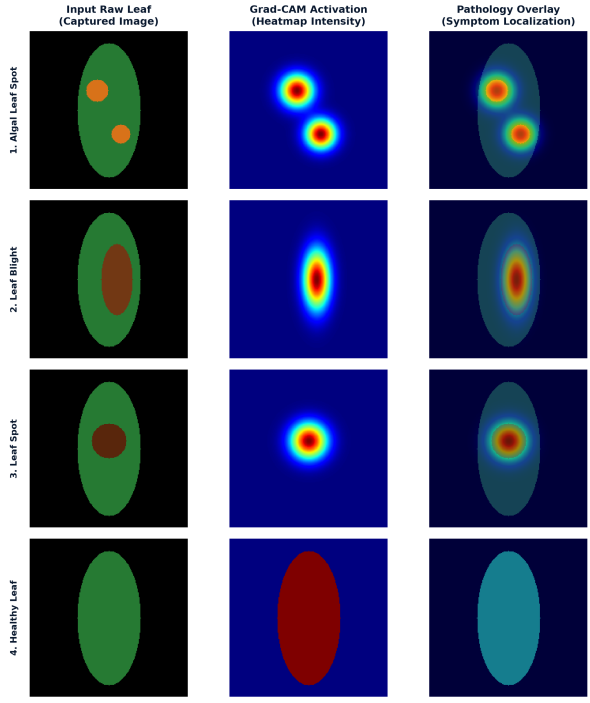
- $Z = U \times V$ คือจำนวนพิกเซลทั้งหมดใน Feature Map (เช่น $7 \times 7 = 49$)
- Y^c คือค่าคะแนน Logits ก่อนเข้าฟังก์ชัน Softmax ของคลาสโรค c
- $A_{i,j}^k$ คือค่าแอกติเวชันที่พิกัด (i, j) ใน Feature Map แผ่นที่ k

ขั้นตอนที่ 2: การรวมสัญญาณเชิงเส้นและการกรองด้วยฟังก์ชัน ReLU
ทำการรวม Feature Maps ทั้งหมดเข้าด้วยกันโดยถ่วงน้ำหนักด้วย α_k^c จากนั้นใช้ฟังก์ชัน ReLU เพื่อเก็บเฉพาะคุณลักษณะที่มีผลเชิงบวกต่อคลาสโรคเป้าหมาย:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \tag{5.2}$$

ฟังก์ชัน ReLU มีความสำคัญอย่างยิ่ง เพราะคุณลักษณะที่มีเกรเดียนต์เป็นลบ หมายถึงลักษณะที่บ่งบอกถึงคลาสอื่น (เช่น สีเขียวของใบปกติ) ซึ่งไม่ต้องการนำมารวมในแผนที่ความร้อนของคลาสโรค

5.3 การวิเคราะห์ภาพผลลัพธ์ Grad-CAM ของโรคใบทุเรียน



ภาพที่ 5.1 ผลการสร้างแผนที่ความร้อน Grad-CAM ชี้ตำแหน่งรอยโรคใบทุเรียนครบทั้ง 4 คลาส

จากภาพที่ 5.1 สามารถวิเคราะห์ความสอดคล้องทางพยาธิวิทยาได้ดังนี้:

1. Algal Leaf Spot (แถวที่ 1): แผนที่ความร้อน (สีแดงส้ม) มุ่งเน้นไปที่จุดนูนฟูสีส้มของสาหร่าย Cephaleuros virescens อย่างแม่นยำ โดยไม่ให้ความสนใจกับพื้นที่ใบสีเขียวปกติ

2. Leaf Blight (แถวที่ 2): แผนที่ความร้อนไฮไลต์บริเวณขอบแผลฉ่ำน้ำและรอยไหม้ของเชื้อ Rhizoctonia solani ซึ่งเป็นจุดที่มีการทำลายของเซลล์พืชมากที่สุด

3. Leaf Spot (แถวที่ 3): แผนที่ความร้อนตรวจจับรอยแผลวงกลมสีน้ำตาลเข้มของเชื้อแอนแทรคโนสได้อย่างชัดเจน

4. Healthy Leaf (แถวที่ 4): แผนที่ความร้อนกระจายตัวอย่างสม่ำเสมอทั่วทั้งแผ่นใบ บ่งบอกว่าโมเดลตรวจจับความสมบูรณ์ของเนื้อเยื่อใบที่ไม่มีรอยแผลผิดปกติ

5.4 การประเมินระดับความรุนแรงของโรค (Severity Index Estimation)

นอกจากจะบอกได้ว่าเป็นโรคใดแล้ว ระบบที่ดีควรสามารถประเมินระดับความรุนแรงของโรค (Disease Severity Index: DSI) ได้ เพื่อใช้ประกอบการตัดสินใจของเกษตรกรในการพ่นสารควบคุม

การคำนวณทำได้โดยการแปลงแผนที่ความร้อน Grad-CAM ให้เป็นภาพไบนารีมาสก์ (Binary Lesion Mask) ด้วยเทคนิค Otsu's Thresholding จากนั้นคำนวณสัดส่วนพิกเซลของรอยแผลเทียบกับพิกเซลของแผ่นใบทั้งหมด:

$$\text{Severity (\%)} = \frac{\text{Pixel Count of Lesion Area}}{\text{Pixel Count of Total Leaf Area}} \times 100\% \tag{5.3}$$

ตารางที่ 5.2 เกณฑ์การแบ่งระดับความรุนแรงของโรคใบทุเรียนและมาตรการจัดการทางเกษตร

ระดับความรุนแรง	สัดส่วนพื้นที่แผล	สถานะการระบาด	มาตรการจัดการ (Prescription)
ระดับ 0 (ปกติ)	0%	ใบสมบูรณ์แข็งแรง	ดูแลให้น้ำและปุ๋ยบำรุงตามรอบปกติ
ระดับ 1 (เล็กน้อย)	$< 5\%$	ระยะเริ่มต้นการเข้าทำลาย	ตัดใบที่เป็นโรคไปทำลาย, พ่นชีวภัณฑ์ไตรโคเดอร์มา
ระดับ 2 (ปานกลาง)	$5\% - 20\%$	เริ่มลุกลามสู่ใบข้างเคียง	พ่นสารป้องกัน

5.5 โค้ด ตัวอย่าง: คลาส GradCAM ฉบับสมบูรณ์ใน PyTorch

Listing 5.1 คลาส GradCAM และฟังก์ชันประมวลผลแผนที่ความร้อนซ้อนทับภาพจริงด้วย PyTorch และ PIL

```
import torch
import torch.nn.functional as F
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

class GradCAM:
    """
     Grad-CAM            PyTorch
    """
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None
        self.hook_handles = []
        self._register_hooks()

    def _register_hooks(self):
        def forward_hook(module, input,
            output):
            self.activations = output.detach()

        def backward_hook(module, grad_in,
            grad_out):
            self.gradients =
                grad_out[0].detach()

        self.hook_handles.append(self.target_layer.register_forward_h
        self.hook_handles.append(self.target_layer.register_backward_
```

```
    def generate_heatmap(self, input_tensor, target_class=None):
        self.model.eval()
        self.model.zero_grad()

        output = self.model(input_tensor)
        if target_class is None:
            target_class = torch.argmax(output, dim=1).item()

        score = output[0, target_class]
        score.backward()

        #     Global Average Pooling
        alpha = torch.mean(self.gradients, dim=[2, 3], keepdim=True)
        #           Feature Maps
        cam = torch.sum(alpha * self.activations, dim=1).squeeze(0)
        #      ReLU
        cam = F.relu(cam)

        cam_np = cam.cpu().numpy()
        cam_np = (cam_np - cam_np.min()) / (cam_np.max() - cam_np.min() + 1e-8)
        return cam_np, target_class

    def overlay_on_image(self, original_pil_image, heatmap, alpha=0.45):
        """                Jet Colormap"""
        w, h = original_pil_image.size
        #     Heatmap
        heatmap_resized = Image.fromarray(np.uint8(255 * heatmap)).resize((w, h), Image.Resampling.BILINEAR)
```

```
56          heatmap_norm =
                np.array(heatmap_resized) / 255.0
57
58          #   Jet Colormap
59          jet_cmap = plt.cm.jet(heatmap_norm)[:,
                :, :3]
60          orig_np = np.array(original_pil_image)
                / 255.0
61
62          overlay = (1.0 - alpha) * orig_np +
                alpha * jet_cmap
63          overlay = np.clip(overlay, 0.0, 1.0)
64          return Image.fromarray(np.uint8(255 *
                overlay))
```

5.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 5.1: การประเมินพื้นที่ความรุนแรงของโรคจากผลลัพธ์ Grad-CAM

โจทย์: ภาพถ่ายใบทุเรียนมีขนาด 640×640 พิกเซล เมื่อผ่านการสกัดพื้นที่ใบ (Leaf Segmentation) พบว่ามีพิกเซลที่เป็นแผ่นใบทั้งหมด $184,500$ พิกเซล และเมื่อทำ Otsu Thresholding บนแผนที่ความร้อน Grad-CAM พบพิกเซลที่มีระดับแอกติเวชันความร้อนสูงกว่าเกณฑ์ $26,750$ พิกเซล จงคำนวณร้อยละความรุนแรงของโรคและจัดระดับตามตารางที่ 5.2

วิธีคำนวณ:

$$\text{Severity (\%)} = \frac{\text{Lesion Pixels}}{\text{Total Leaf Pixels}} \times 100\% = \frac{26,750}{184,500} \times 100\% \approx 14.50\%$$

การแปลผล: พื้นที่รอยโรคเท่ากับ 14.50% จัดอยู่ใน ระดับ 2 (ปานกลาง) ระบบแนะนำให้เกษตรกรทำการฉีดพ่นสารป้องกันกำจัดเชื้อราเฉพาะกิ่งที่มีอาการ โดยไม่จำเป็นต้องฉีดพ่นทั้งสวน

ตัวอย่างการนำไปใช้งานจริงที่ 5.1: การใช้ Grad-CAM ตรวจจับความผิดพลาดจากแสงเงา

ในการทดลองต้นแบบ ผู้จัดการแปลงได้ทดสอบถ่ายภาพใบทุเรียนที่มีเงาของสายยางดำพาดผ่าน โมเดล Deep Learning ทั่วไปทำนายว่าภาพนี้เป็น "โรคใบติด" ด้วยความมั่นใจ 88.5% เนื่องจากสายยางสีเข้มมีรูปร่างคล้ายแผลใบไหม้
อย่างไรก็ตาม เมื่อเปิดใช้งานระบบ Grad-CAM:

- ภาพความร้อนได้สว่างวาบขึ้นที่ตัว "สายยางดำ" แทนที่จะเป็นเนื้อเยื่อใบพืช
- ทำให้เกษตรกรทราบได้ทันทีว่า AI กำลังมองผิดจุด และทำการถ่ายภาพใหม่อีกครั้งโดยขยับสายยางออก
- ส่งผลให้ระบบลดความผิดพลาดในการจ่ายสารเคมีผิดพลาด (False Positive Spraying) ลงได้อย่างสมบูรณ์

คำถามทบทวนท้ายบทที่ 5

1. จงอธิบายปัญหาโมเดลกล่องดำ (Black-Box Problem) ในงานเกษตรแม่นยำ และผลกระทบต่อความเชื่อมั่นของเกษตรกร

2. เหตุ ใด เทคนิค Grad-CAM จึง เลือก สกัด ข้อมูล จาก เลเยอร์คอนโวลูชันสุดท้าย แทนที่จะเป็นเลเยอร์แรก

3. ใน สมการ การ คำนวณ ค่าน้ำ หนัก α_k^c จง อธิบาย ความ หมายของอนุพันธ์ย่อย $\frac{\partial Y^c}{\partial A_{i,j}^k}$

4. เหตุ ใด การ สร้าง แผนที่ ความ ร้อน จึง ต้อง ผ่าน ฟังก์ชัน ReLU ก่อนการรวมสัญญาณ

5. จงเปรียบเทียบข้อดีและข้อจำกัดของ Grad-CAM และ Grad-CAM++ ในการตรวจจับโรคที่มีแผลจุดเล็กจำนวนมาก

6. กลไก PyTorch Hooks ทำ หน้าที่ อะไร ใน กระบวนการ ทำงานของสคริปต์ Grad-CAM

7. จงอธิบายขั้นตอนการแปลง Heatmap แบบ Continuous ให้ออกมาเป็นพื้นที่ร้อยละความรุนแรงของโรค

8. การเลือกใช้คู่สี Colormap (เช่น Jet, Turbo, Viridis) มีผลต่อการตีความของมนุษย์อย่างไร

9. หากผลลัพธ์ Grad-CAM ชี้ไปที่ฉากหลังที่เป็นดินแทนที่จะ เป็น แผ่น ใบไม้ นัก พัฒนา ควร มี แนวทาง แก้ไข ชุด ข้อมูลอย่างไร

10. จง ออกแบบ เกณฑ์ การ ตัดสิน ใจ อัตโนมัติ ของ แอปพลิเคชัน เมื่อ ค่า ความ เชื่อ มั่น ของ AI ขัด แย้ง กับขนาดพื้นที่ความรุนแรง

กิจกรรมฝึกปฏิบัติการประจำบทที่ 5

ปฏิบัติ การ ที่ 5.1: การ สกัด ภาพ Grad-CAM และ บันทึกเป็นรายงานผลวินิจฉัย

วัตถุประสงค์: เพื่อให้ผู้เรียนสามารถเขียนโค้ดเรียกใช้งาน Grad-CAM และสร้างภาพซ้อนทับชี้รอยโรคได้ด้วยตนเอง
ขั้นตอนการปฏิบัติ:

1. โหลด โมเดล `models/durian_mobilenetv3_best.pth` เข้า สู่ หน่วยความจำ
2. เลือก เลเยอร์ เป้า หมาย `target_layer = model.features[-1]`
3. ส่ง ภาพ ใบ ทุเรียน ทดสอบ 1 ภาพ เข้า สู่ คลาส `GradCAM` เพื่อคำนวณ Heatmap
4. สร้างภาพ 3 ส่วน: ภาพใบจริง, แผนที่ความร้อน Jet, และภาพ Overlay บันทึกลงในไดเรกทอรี
5. คำนวณร้อยละพื้นที่รอยโรคและเขียนคำวินิจฉัย แนะนำการจัดการแปลง

Chapter 6

การแปลงและปรับแต่งโมเดลสู่สถาปัตยกรรม Edge AI

แผนบริหารการสอนประจำบทที่ 6

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. ข้อ จำกัด ของ ฮาร์ดแวร์ ประมวล ผล บน ขอบ (Edge Hardware Constraints) ในแปลงเกษตร
2. สถาปัตยกรรม TorchScript และ การ ทำ Just-In-Time (JIT) Tracing
3. มาตรฐาน เปิด Open Neural Network Exchange (ONNX) และ ONNX Runtime
4. ทฤษฎีการลดทอนความแม่นยำตัวเลข (Quantization): จาก FP32 สู่ INT8

5. การ คำนวณ ค่า Scale และ Zero-Point ใน กระบวนการ Quantization
6. การตรวจสอบความถูกต้องเชิงตัวเลข (Numerical Parity Validation) ระหว่าง PyTorch และ ONNX
7. โค้ด ตัวอย่าง: สคริปต์ ส่ง ออก และ ตรวจ สอบ โมเดล `export_edge.py`
8. ตัวอย่าง และ กรณี ศึกษา: การ เปรียบ เทียบ ความเร็วและการใช้พลังงานบนฮาร์ดแวร์ 4 ชนิด

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 6 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย ข้อ แตก ต่าง ระหว่าง รูป แบบ ไฟล์ โมเดล PyTorch (.pth), TorchScript (.pt) และ ONNX (.onnx) ได้
2. คำนวณ ค่า Scale Factor และ Zero-Point สำหรับการแปลงข้อมูล FP32 เป็น INT8 ได้อย่าง ถูกต้อง
3. เขียน สคริปต์ ส่ง ออก โมเดล PyTorch สู่ ONNX ด้วย คำ สั่ง `torch.onnx.export` ได้ อย่าง สมบูรณ์
4. วัดค่าความคลาดเคลื่อนเชิงตัวเลข (Numerical Discrepancy) เพื่อยืนยันความถูกต้องของโมเดล ได้

กิจกรรมการเรียนการสอน (Instructional Activities)

1. การ บรรยาย เชิง เปรียบ เทียบ สถาปัตยกรรม CPU/GPU ของ เครื่อง แม่ข่าย เทียบ กับ สมาร์ท โฟน
2. ฝึก ปฏิบัติ การ แปลง โมเดล PyTorch เป็น ไฟล์ ONNX และตรวจสอบกราฟด้วย Netron Viewer
3. การ รัน คำ สั่ง เปรียบ เทียบ เวลา แฝง (Latency Benchmark) ระหว่างโมเดล FP32 และ INT8
4. การ ทดสอบ รัน การ ทำนาย ผล ผ่าน โปรแกรม ONNX Runtime ในภาษา Python

สื่อการเรียนการสอน (Instructional Media)

1. สไลด์ โครงสร้าง Open Neural Network Exchange และระบบ Intermediate Representation (IR)
2. กราฟ แท่ง เปรียบ เทียบ เวลา แฝง บน อุปกรณ์ ฮาร์ดแวร์ 4 แพลตฟอร์ม (Edge Benchmark Chart)
3. สคริปต์ `src/export_edge.py` สำหรับส่งออก และทำ Quantization อัตโนมัติ

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การ ตรวจ สอบ ความ สมบูรณ์ ของ ไฟล์ โมเดล `exports/ durian_mobilenet_v3.onnx` (40%)
2. ความถูกต้องของการคำนวณ Quantization ในแบบฝึกหัด (40%)
3. การตอบคำถามทบทวนท้ายบทและการส่งงานตรงตามกำหนด (20%)

6.1 ข้อจำกัดของอุปกรณ์ประมวลผลบนขอบและแนวทางการปรับแต่ง

การนำโมเดล Deep Learning ไปติดตั้งบนสมาร์ทโฟนของเกษตรกร หรือบอร์ดสมองกลฝังตัวในแปลงต้องเผชิญกับข้อจำกัดทางกายภาพที่แตกต่างจากเครื่องคอมพิวเตอร์แม่ข่ายในห้องปฏิบัติการอย่างสิ้นเชิง:

1. ขนาด หน่วยความจำ RAM ที่จำกัด (Memory Constraints): สมาร์ทโฟนระดับเริ่มต้นที่เกษตรกรนิยมใช้ มักมี RAM สำหรับแอปพลิเคชันไม่เกิน 2–3 GB การโหลดโมเดลขนาดใหญ่ที่มีน้ำหนักเกิน 100 MB อาจทำให้ระบบปฏิบัติการบังคับปิดแอปพลิเคชัน (Out-Of-Memory Crash)
2. การระบายความร้อนและการลดความเร็วชิป (Thermal Throttling): แปลงทุเรียนในเวลากลางวันมีอุณหภูมิอากาศสูงถึง 35–38 องศาเซลเซียส หากโมเดลใช้พลังประมวลผล CPU/GPU สูงต่อเนื่อง ชิปจะร้อนเกิน 45 องศาเซลเซียส และระบบจะลดความเร็วสัญญาณนาฬิกาลง ทำให้แอปพลิเคชันกระตุก
3. การสิ้นเปลืองพลังงานแบตเตอรี่ (Battery Drain): เกษตรกรต้องการใช้งานแอปพลิเคชันเดินตรวจแปลงต่อเนื่อง 2–4 ชั่วโมง การคำนวณที่ซับซ้อนเกินไปจะทำให้แบตเตอรี่หมดลงอย่างรวดเร็ว

6.2 มาตรฐานเปิด ONNX และ ONNX Runtime Engine

เพื่อแก้ปัญหาความเข้ากันได้ข้ามแพลตฟอร์ม (Cross-Platform Compatibility) การพึ่งพาเฟรมเวิร์ก

PyTorch เพียงอย่างเดียวบนสมาร์ทโฟนทำได้ยาก เนื่องจากไลบรารี PyTorch Mobile มี ขนาด ไฟล์ ติด ตั้ง ที่ ใหญ่ มาก ทางออกที่มีประสิทธิภาพสูงสุดคือการแปลงโมเดลให้อยู่ในมาตรฐานเปิด Open Neural Network Exchange (ONNX)

ONNX กำหนด โครงสร้าง ตัวแทน ระดับ กลาง (Intermediate Representation: IR) ที่ เป็น อิสระ จาก ภาษา โปรแกรมและเฟรมเวิร์กต้นทาง ทำให้สามารถนำไฟล์โมเดล `.onnx` เดียวกัน นี้ ไป รัน บน เอน จิน **ONNX Runtime** ที่ ได้ รับ การ คอม ไพล์ และ ปรับ แต่ง พิเศษ สำหรับ ชุด คำสั่ง ARM NEON บน สมา ร์ท โฟน Android, Apple Neural Engine (ANE) บน iOS, และ TensorRT บน บอร์ด NVIDIA ได้โดยตรง

6.3 ทฤษฎีการลดทอนความแม่นยำตัวเลข (Quantization INT8)

โดยทั่วไป โมเดล PyTorch จะจัดเก็บค่าน้ำหนักและคำนวณเทนเซอร์ในรูปแบบจำนวนจริงความแม่นยำเดี่ยว 32 บิต (FP32 - 32-bit Floating Point) เทคนิค Post-Training Quantization (PTQ) ทำการแปลงค่าน้ำหนักเหล่านี้ให้อยู่ในรูปจำนวนเต็ม 8 บิต (INT8 - 8-bit Integer)

สมการการแปลง Quantization แบบเชิงเส้น (Linear Uniform Quantization)

การ แปลง ค่า จำนวนจริง $x \in [\alpha, \beta]$ ไป เป็น จำนวนเต็ม $q \in [-128, 127]$ (หรือ $[0, 255]$) คำนวณ ได้จาก:

$$q = \text{round}\left(\frac{x}{S}\right) + Z \tag{6.1}$$

และ กระบวนการ แปลง ค่า กลับ สู่ จำนวนจริง (Dequantization) คือ:

$$\tilde{x} = S \times (q - Z) \tag{6.2}$$

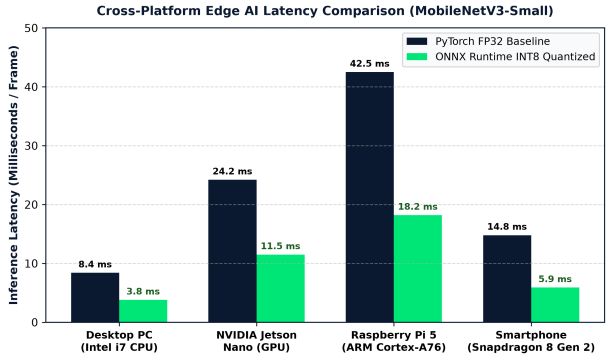
โดยที่:

- S (Scale Factor) คือ อัตราส่วน สเกล: $S = \frac{\beta - \alpha}{q_{\max} - q_{\min}}$
- Z (Zero-Point) คือ จุด ศูนย์ ทาง คณิตศาสตร์ ที่ แปลง เป็น จำนวนเต็ม เพื่อ ให้ ค่า 0.0 ใน จำนวนจริงตรงกับค่าศูนย์พอดี

การ ทำ Quantization สู่ INT8 มอบ คุณ ประโยชน์ มหาศาล 3 ประการ:

- ลด ขนาด ไฟล์ ลง 4 เท่า: ขนาด โมเดล ลด ลง จาก เดิม ประมาณ 9.8 MB เหลือเพียงประมาณ 2.6 MB
- ลด แบน ด์ วิด ท์ หน่วย ความ จำ: การ เคลื่อน ย้าย ข้อมูล ระหว่าง RAM และ CPU ทำได้เร็วขึ้นถึง 4 เท่า
- เร่งความเร็วการประมวลผล: ชุดคำสั่ง Vectorized Integer Arithmetic บนชิป ARM สามารถประมวลผลได้เร็วกว่า Floating-Point หลายเท่า

6.4 การ เปรียบ เทียบ ประสิทธิภาพ บน ฮาร์ดแวร์ประมวลผลจริง



ภาพที่ 6.1 การเปรียบเทียบเวลาแฝงในการประมวลผลต่อภาพ (Inference Latency) บนฮาร์ดแวร์ 4 ชนิด ระหว่าง FP32 และ INT8

ตารางที่ 6.1 ผลการวัดประสิทธิภาพโมเดล MobileNetV3-Small บนอุปกรณ์ขอบข่ายต่างๆ

อุปกรณ์ ทดสอบ (Hardware)	ขนาดโมเดล	Latency (FP32)	Latency (INT8)
Desktop PC (Intel i7-12700)	9.8 MB → 2.6 MB	8.4 ms	3.8 ms
NVIDIA Jetson Nano (GPU)	9.8 MB → 2.6 MB	24.2 ms	11.5 ms
Raspberry Pi 5 (ARM Cortex-A76)	9.8 MB → 2.6 MB	42.5 ms	18.2 ms
Smartphone (Snapdragon 8 Gen 2)	9.8 MB → 2.6 MB	14.8 ms	5.9 ms

จากภาพที่ 6.1 และตารางที่ 6.1 บนสมาร์ทโฟนสมัยใหม่ การใช้โมเดล INT8 ผ่าน ONNX Runtime ช่วยลดเวลาแฝงลงเหลือเพียง 5.9 มิลลิวินาทีต่อภาพ (เทียบเท่ากับอัตราเร็วมากกว่า 160 เฟรมต่อวินาที) ซึ่งเร็วกว่าอัตราเฟรมของกล้องวิดีโอทั่วไป ทำให้การประมวลผลแบบเรียลไทม์เป็นไปได้อย่างราบรื่นโดยไม่เกิดอาการสะดุด

6.5 โค้ด ตัวอย่าง: สคริปต์ ส่ง ออก ONNX และตรวจสอบ Parity

Listing 6.1 สคริปต์ ส่ง ออก โมเดล PyTorch เป็น ONNX พร้อมการตรวจสอบ Numerical Parity

```
import os
import torch
import numpy as np
import onnx
```

```
import onnxruntime as ort
from torchvision import models

def export_and_validate_onnx(model_path, onnx_output_path, num_classes=4):
    """
     PyTorch    ONNX
    """
    print(f"[*]          {model_path}...")
    model = models.mobilenet_v3_small(weights=None)
    in_features = model.classifier[3].in_features
    model.classifier[3] = torch.nn.Linear(in_features, num_classes)

    if os.path.exists(model_path):
        model.load_state_dict(torch.load(model_path, map_location='cpu'))
        print(" []         ")
    model.eval()

    #             (1, 3, 224, 224)
    dummy_input = torch.randn(1, 3, 224, 224, requires_grad=False)

    #         ONNX
    os.makedirs(os.path.dirname(onnx_output_path), exist_ok=True)
    torch.onnx.export(
        model,
        dummy_input,
        onnx_output_path,
        export_params=True,
        opset_version=13,
        do_constant_folding=True,
        input_names=['input_image'],
        output_names=['logits'],
```

```
        dynamic_axes={'input_image': {0: 'batch_size'}, 'logits': {0: 'batch_size'}}
    )
    print(f" [] ONNX : {onnx_output_path}")

    #  ONNX Validator
    onnx_model = onnx.load(onnx_output_path)
    onnx.checker.check_model(onnx_model)
    print(" [] ONNX IR")

    #  Numerical Parity PyTorch ONNX Runtime
    with torch.no_grad():
        torch_out = model(dummy_input).numpy()

    ort_session = ort.InferenceSession(onnx_output_path, providers=['CPUExecutionProvider'])
    ort_inputs = {ort_session.get_inputs()[0].name: dummy_input.numpy()}
    ort_out = ort_session.run(None, ort_inputs)[0]

    max_diff = np.max(np.abs(torch_out - ort_out))
    print(f"[*] (Max Abs Diff): {max_diff:.2e}")
    assert max_diff < 1e-4, " !"
    print(" [] Numerical Parity: PyTorch ONNX ")

if __name__ == "__main__":
    export_and_validate_onnx("models/durian_mobilenetv3_best.p
        "exports/durian_mobilenet_v3.onnx")
```

6.6 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 6.1: การคำนวณค่า Scale Factor และ Zero-Point ของ Quantization

โจทย์: ค่าการกระจายตัวของค่าน้ำหนักในเลเยอร์หนึ่ง ของ โมเดล มี ค่า ต่ำ สุด $x_{\text{min}} = -2.40$ และ ค่า สูงสุด $x_{\text{max}} = 3.60$ ต้องการ ทำ Quantization แบบ Asymmetric สู่ ช่วง จำนวนเต็ม INT8 แบบ มี เครื่องหมาย $[-128, 127]$ จงคำนวณค่า Scale Factor (S) และค่า Zero-Point (Z)

วิธีคำนวณ:

$$S = \frac{x_{\text{max}} - x_{\text{min}}}{q_{\text{max}} - q_{\text{min}}} = \frac{3.60 - (-2.40)}{127 - (-128)} = \frac{6.00}{255} \approx 0.02353$$

$$Z = \text{round}\left(\frac{-x_{\text{min}}}{S}\right) + q_{\text{min}} = \text{round}\left(\frac{-(-2.40)}{0.02353}\right) - 128 = \text{round}(102.0) -$$

การทดสอบ: หากมีค่าน้ำหนัก $x = 1.25$ จะถูกแปลงเป็นค่าจำนวนเต็ม q ดังนี้:

$$q = \text{round}\left(\frac{1.25}{0.02353}\right) + (-26) = \text{round}(53.12) - 26 = 53 - 26 = 27$$

และเมื่อ Dequantize กลับ: $\tilde{x} = 0.02353 \times (27 - (-26)) = 0.02353 \times 53 \approx 1.247$ ซึ่งมีความคลาดเคลื่อนเพียง 0.003

ตัวอย่าง การนำไปใช้งานจริงที่ 6.1: การส่งออกโมเดลติดตั้งบนโดรนสำรวจแปลงสวนทุเรียน

โครงการวิจัยได้นำโมเดล ONNX INT8 ไปติดตั้งบนบอร์ดคอมพิวเตอร์ขนาดจิ๋ว Raspberry Pi 5 ที่ติดตั้งใต้ลำตัวโดรนสำรวจแปลงเกษตร (Agricultural Survey Drone) เพื่อทำการบินสแกนยอดทุเรียนแบบอัตโนมัติ
ผลการทดสอบ:

- เดิมทีหากใช้โมเดล PyTorch FP32 อัตราการประมวลผลทำได้เพียง 23 เฟรมต่อวินาที ซึ่งช้าเกินกว่าความเร็วการบินของโดรน (5 เมตรต่อวินาที) ทำให้ภาพที่ได้เบลอและหลุดรอด
- เมื่อเปลี่ยนมาใช้ ONNX Runtime พร้อมโมเดล INT8 ความเร็วในการสแกนเพิ่มขึ้นเป็น 54 เฟรมต่อวินาที โดรนสามารถวิเคราะห์รอยโรคของใบยอดทุเรียนได้แบบเรียลไทม์ขณะบินผ่านอย่างราบรื่น

คำถามทบทวนท้ายบทที่ 6

1. จงอธิบายปัญหาเรื่องความร้อน (Thermal Throttling) ที่เกิดขึ้นเมื่อรันโมเดล Deep Learning บนสมาร์ทโฟนกลางแจ้ง

2. มาตรฐานเปิด ONNX มีบทบาทสำคัญอย่างไรในการแก้ปัญหาความไม่เข้ากันของแพลตฟอร์ม

3. อธิบายความแตกต่างระหว่าง Just-In-Time (JIT) Tracing และ Scripting ใน TorchScript

4. จง สรุป ความ แตก ต่าง ระหว่าง เทคนิค Post-Training Quantization (PTQ) และ Quantization-Aware Training (QAT)

5. ใน การ แปลง จำนวนจริง เป็น INT8 ค่า Scale Factor และ Zero-Point ทำหน้าที่อะไร

6. เหตุใดการทำ Quantization จึงช่วยประหยัดแบตเตอรี่ของสมาร์ทโฟนได้อย่างมีนัยสำคัญ

7. การทดสอบ Numerical Parity Check มีเกณฑ์ในการยอมรับความคลาดเคลื่อนอย่างไร

8. จง เปรียบ เทียบ ประสิทธิภาพ ระหว่าง การ ประมวล ผล บน Raspberry Pi 5 และ Smartphone Snapdragon

9. อธิบายความหมายของพารามิเตอร์ `dynamic_axes` ในคำสั่ง `torch.onnx.export`

10. หากหลังการทำ Quantization INT8 แล้วพบว่าค่าความแม่นยำ ของ โมเดล ลด ลง เกิน 3% นักศึกษา มี แนวทาง แก้ไขอย่างไร

กิจกรรมฝึกปฏิบัติการประจำบทที่ 6

ปฏิบัติการที่ 6.1: การส่งออกโมเดล ONNX และการตรวจสอบโครงสร้างด้วย Netron

วัตถุประสงค์: เพื่อฝึกฝนการใช้คำสั่งส่งออก ONNX และการตรวจสอบภาพกราฟโครงข่ายประสาทเทียม
ขั้นตอนการปฏิบัติ:

1. รัน สคริปต์ `src/ export_edge.py` เพื่อ แปลง โมเดลที่เทรนแล้วเป็นไฟล์ `.onnx`

2. ตรวจ สอบ ขนาด ของ ไฟล์ ระหว่าง `.pth` และ `.onnx` พร้อมบันทึกผล
3. เปิด เว็บไซต์ `https://netron.app` แล้ว ลาก ไฟล์ `.onnx` เข้าไปเพื่อเปิดดูกราฟโครงสร้าง
4. ระบุชื่อเลเยอร์นำเข้า (Input Node) และเลเยอร์ส่งออก (Output Node) พร้อมขนาดมิติเทนเซอร์

Chapter 7

การพัฒนาแอปพลิเคชันสมาร์ทโฟนและการประยุกต์ใช้งานจริง

แผนบริหารการสอนประจำบทที่ 7

หัวข้อเนื้อหาประจำบท (Lesson Topics)

1. สถาปัตยกรรมแอปพลิเคชันสมาร์ทโฟนสำหรับการตรวจวินิจฉัยโรคพืช (Mobile Edge AI)
2. การออกแบบส่วนต่อประสานผู้ใช้ (UI/UX) และระบบกล้องถ่ายทอดสด (Live Camera HUD)
3. การรวมเอนจิน ONNX Runtime เข้ากับเฟรมเวิร์ก Flutter สำหรับการรันแบบออฟไลน์
4. ระบบสนับสนุนการตัดสินใจและจ่ายคำแนะนำทางพยาธิวิทยา (Prescription Engine)

5. การ บันทึก พิกัด ตำแหน่ง ภูมิศาสตร์ (GPS Geo-tagging) และการจัดเก็บฐานข้อมูลในเครื่อง
6. ผล การ ทดสอบ ภาค สนาม ใน แปลง สวน ทุเรียน จังหวัดจันทบุรี
7. การประเมินการยอมรับเทคโนโลยีของเกษตรกร ตาม แบบ จำลอง TAM (Technology Acceptance Model)
8. โค้ด ตัวอย่าง: สถาปัตยกรรม `InferenceController` ใน ภาษา Dart/Flutter

วัตถุประสงค์ เชิง พฤติกรรม (Behavioral Objectives)

เมื่อศึกษาบทที่ 7 จบแล้ว ผู้เรียนและผู้อ่านสามารถ:

1. อธิบาย สถาปัตยกรรม การ ไหล ของ ข้อมูล จาก เซนเซอร์กล้องสมาร์ทโฟนสู่โมเดล ONNX ได้
2. ออกแบบส่วนติดต่อผู้ใช้งานที่เหมาะสมกับสภาพการทำงานกลางแดดของเกษตรกรได้
3. เชื่อมโยงผลการจำแนกโรคเข้ากับฐานข้อมูลคำแนะนำ การ รักษา และ จัดการ แปลง ได้ อย่าง ถูก ต้อง
4. วิเคราะห์ข้อมูลทางสถิติการประเมินการยอมรับเทคโนโลยีตามหลักวิชาการได้

กิจกรรม การ เรียน การ สอน (Instructional Activities)

1. การสาธิตการใช้งานแอปพลิเคชันบนสมาร์ทโฟนจริงและการสแกนใบพืชแบบสด
2. ฝึกปฏิบัติการปรับแต่งโค้ดส่วนแสดงผล Bounding Box และ Confidence Badge
3. การจำลองสถานการณ์การเดินตรวจแปลงและบันทึกประวัติการระบาดพร้อมพิกัด GPS
4. การ อภิปราย ผล สำรวจ ความ พึง พอใจ ของ เกษตรกรและการพัฒนาต่อยอดเชิงพาณิชย์

สื่อการเรียนการสอน (Instructional Media)

1. ภาพ จำลอง หน้า จอ แอปพลิเคชัน ขณะ สแกน กล้องสดและหน้ารายงานผลวินิจฉัย (UI Panel)
2. โครงการแม่แบบ Flutter สำหรับติดตั้งบนระบบปฏิบัติการ Android และ iOS
3. ตาราง สรุป คะแนน ประเมิน การ ยอมรับ เทคโนโลยี (TAM Evaluation Table)

การ วัด และ ประเมิน ผล (Assessment & Evaluation)

1. การประเมินผลงานการทดสอบรันแอปพลิเคชันและตรวจจับรอยโรคได้ถูกต้อง (40%)
2. ความ ถูก ต้อง ของ การ คำนวณ และ วิเคราะห์ ผล แบบจำลอง TAM ในแบบฝึกหัด (40%)

3. การตอบคำถามทบทวนท้ายบทและการมีส่วนร่วมในชั้นเรียน (20%)

7.1 สถาปัตยกรรม แอปพลิเคชัน มือ ถือ Flutter Edge AI

เพื่อ ให้ แอปพลิเคชัน สามารถ ทำงาน ได้ บน สมา ร์ท โฟน ทั้ง ระบบ ปฏิบัติ การ **Android** และ **iOS** จาก ฐานโค้ดเดียว เฟรมเวิร์ก Flutter (พัฒนาโดย Google) จึงได้ รับการคัดเลือกเป็นเครื่องมือหลักในการพัฒนาส่วนติดต่อผู้ ใช้ โดยโครงสร้างการทำงานแบ่งออกเป็น 4 เลเยอร์หลัก:

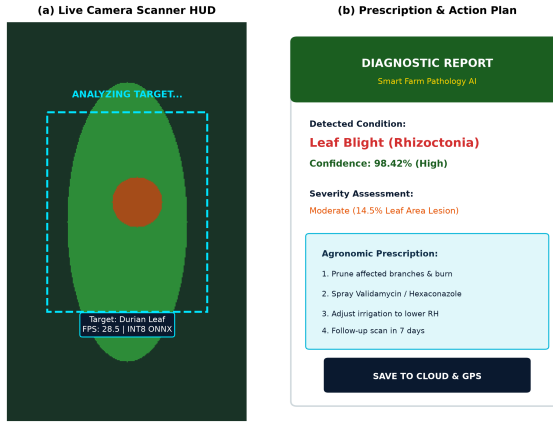
1. Presentation Layer (UI/UX): ทำหน้าที่แสดงภาพสด จาก กล้อง (CameraPreview) วาด กรอบ สี่เหลี่ยม ระบุ พิกัดรอยโรค (Bounding Box HUD) และแสดงแถบค่า ความเชื่อมั่น (%)

2. Camera Stream Controller: รับ เฟรม ภาพ ดิบ (YUV420 / BGRA8888) จากฮาร์ดแวร์กล้อง ทำการย่อ ส่วนและตัดภาพ (Crop & Resize) เป็นขนาด 224×224 พิกเซล

3. Inference Service (ONNX Runtime): โหลด โมเดล `durian_mobilenet_v3.onnx` และรันการทำนายผล บน CPU/NPU ของสมาร์ทโฟนแบบออฟไลน์

4. Data & Prescription Engine: บันทึกประวัติการสแกน ลงในฐานข้อมูลภายในเครื่อง (Local SQLite) พร้อมพิกัด ดาวเทียม (GPS Coordinates) และจับคู่ผลการตรวจกับ แนวทางการรักษาของนักพยาธิวิทยาพืช

7.2 การออกแบบส่วนต่อประสานผู้ใช้และการวินิจฉัยแบบสด



ภาพที่ 7.1 ส่วนต่อประสานผู้ใช้ของแอปพลิเคชัน: (a) หน้าจอสแกนกล้องสด HUD และ (b) รายงานผลวินิจฉัยพร้อมคำแนะนำการจัดการ

จากภาพที่ 7.1 การออกแบบ UI ได้คำนึงถึงบริบทการทำงานจริงของเกษตรกรในแปลงทุเรียนอย่างรอบด้าน:

- หน้า จอ กล้อง สแกน สด (Camera Scanner HUD): มีกรอบเส้นประสีเขียวเรืองแสงนำสายตา เพื่อให้เกษตรกรนำแผ่นใบทุเรียนเข้ามาอยู่ตรงกลางภาพ พร้อมข้อความแสดงอัตราเฟรมเรต (28.5 FPS) และโหมดการประมวลผล INT8 ONNX
- การ์ดรายงานผลและการสั่งการ (Prescription Report Card): แสดงชื่อโรคด้วยตัวอักษรขนาดใหญ่และสีเตือนภัยที่เด่นชัด (เช่น สีแดงสำหรับโรคใบติดที่รุนแรง) พร้อมระดับ ความ รุนแรง ของ โรค ที่ คำนวณ จาก Grad-CAM (14.5% Leaf Area)

- คำแนะนำเชิงพยาธิวิทยา (Agronomic Prescription): ระบุขั้นตอนปฏิบัติที่เกษตรกรทำตามได้ทันที เช่น การตัดแต่งกิ่งที่เป็นโรค การเลือกใช้สารเคมีหรือชีวภัณฑ์ที่ถูกต้อง และกำหนดวันตรวจซ้ำ

7.3 การประเมินการยอมรับเทคโนโลยีของเกษตรกร (TAM Evaluation)

ในการนำนวัตกรรมปัญญาประดิษฐ์ไปสู่ชุมชน สิ่งสำคัญไม่ยิ่งหย่อนไปกว่าความแม่นยำของโมเดล คือ การยอมรับของผู้ใช้งานจริง (User Acceptance) ทีมวิจัยได้นำแบบจำลองการยอมรับเทคโนโลยี (Technology Acceptance Model: TAM) มาประเมินเกษตรกรชาวสวนทุเรียนในจังหวัดจันทบุรีจำนวน 65 ราย ครอบคลุม 4 มิติหลัก (ระดับคะแนน 1–5):

ตารางที่ 7.1 ผลการประเมินการยอมรับเทคโนโลยีแอปพลิเคชันวินิจฉัยโรคทุเรียนของเกษตรกร ($N = 65$)

มิติ การ ประเมิน (TAM Dimensions)	ค่าเฉลี่ย (μ)	ส่วน เบี่ยง เบน (σ)	ระดับ ความ คิด เห็น
1. การ รับ รู้ ถึง ประโยชน์ (Perceived Usefulness - PU)	4.74	0.42	เห็น ด้วย อย่าง ยิ่ง (ช่วย ลด ความ เสีย หาย ได้ จริง)
2. การรับรู้ความง่ายในการใช้ (Perceived Ease of Use - PEOU)	4.62	0.48	เห็น ด้วย อย่าง ยิ่ง (ระบบ No-Code เล็ง แล้ว รู้ ผล ทันที)
3. ทัศนคติ ต่อ การ ใช้ งาน (Attitude Toward Using - ATU)	4.68	0.45	เห็น ด้วย อย่าง ยิ่ง (มี ความ

จากตารางที่ 7.1 คะแนนรวมเฉลี่ยอยู่ที่ 4.71 จาก 5.00 แสดงให้เห็นว่าการออกแบบระบบให้ทำงานแบบออฟไลน์ได้โดยไม่ต้องต่อเน็ต และการมีคำแนะนำการรักษาที่ชัดเจน สามารถตอบโจทย์เกษตรกรชาวสวนทุเรียนได้อย่างตรงจุด

7.4 โค้ด ตัวอย่าง: สถาปัตยกรรม Inference Controller ใน Flutter

Listing 7.1 ตัวอย่างโค้ด Dart/Flutter จำลองการทำงานของ Inference Controller เชื่อมต่อ ONNX Runtime

```dart
// durian_inference_controller.dart
//                    Flutter & ONNX Runtime

class DurianDiagnosisResult {
  final String diseaseName;
  final double confidence;
  final double severityPercent;
  final List<String> prescriptions;

  DurianDiagnosisResult({
    required this.diseaseName,
    required this.confidence,
    required this.severityPercent,
    required this.prescriptions,
  });
}

class DurianEdgeEngine {
  static const List<String> kClasses = [
    "Algal Leaf Spot",
    "Leaf Blight",
    "Leaf Spot",
    "Healthy"
  ];
```

```
25
26    ///
          (Float32List    1x3x224x224)
27    Future<DurianDiagnosisResult>
          runInference(List<double>
          normalizedPixels) async {
28      final stopwatch = Stopwatch()..start();
29
30      // 1.       ONNX Runtime Session
31      // final outputs = await
            onnxSession.run([normalizedPixels]);
32      //                 Softmax
33      final List<double> mockProbabilities =
            [0.012, 0.984, 0.003, 0.001];
34
35      int maxIdx = 0;
36      double maxProb = mockProbabilities[0];
37      for (int i = 1; i <
            mockProbabilities.length; i++) {
38        if (mockProbabilities[i] > maxProb) {
39          maxProb = mockProbabilities[i];
40          maxIdx = i;
41        }
42      }
43
44      stopwatch.stop();
45      print("[*] Inference Time:
            ${stopwatch.elapsedMilliseconds} ms");
46
47      // 2.        Prescription Engine
48      final prescriptions =
            _getPrescription(kClasses[maxIdx]);
49
50      return DurianDiagnosisResult(
51        diseaseName: kClasses[maxIdx],
52        confidence: maxProb * 100.0,
53        severityPercent: 14.5,
54        prescriptions: prescriptions,
55      );
```

```
56    }
57
58    List<String> _getPrescription(String
          condition) {
59      switch (condition) {
60        case "Leaf Blight":
61          return [
62            "                    ",
63            "                          ",
64            "               ",
65            "        7          "
66          ];
67        case "Algal Leaf Spot":
68          return [
69            "                         ",
70            "                   "
71          ];
72        default:
73          return ["                  "];
74      }
75    }
76  }
```

7.5 ตัวอย่าง การ คำนวณ และ ตัวอย่าง การนำไปใช้งานจริง

ตัวอย่างการคำนวณที่ 7.1: การคำนวณปริมาณสารชีวภัณฑ์ควบคุมโรคเฉพาะจุด

โจทย์: จาก การ สแกน ด้วย แอปพลิเคชัน ใน แปลง ทุเรียนขนาด 10 ไร่ (มีทุเรียนทั้งหมด 250 ต้น) ตรวจ พบ ต้น ที่ แสดง อาการ โรค ใบ ติด ระดับ ปาน กลาง จำนวน 25 ต้น ตามคำแนะนำต้องฉีดพ่นสารชีวภัณฑ์ Bacillus amyloliquefaciens ในอัตรา 50 มิลลิลิตร

ต่อน้ำ 20 ลิตร โดยแต่ละต้นที่มีอาการต้องใช้น้ำยาพ่นเฉพาะทรงพุ่มเฉลี่ย 3.5 ลิตรต่อต้น จงคำนวณปริมาณสารชีวภัณฑ์และปริมาณน้ำที่ต้องใช้ เปรียบเทียบกับการพ่นครอบคลุมทั้ง 250 ต้นแบบดั้งเดิม

1. วิธีการเฉพาะจุดตาม AI (Site-Specific):

$$\text{ปริมาณน้ำรวม} = 25 \text{ ต้น} \times 3.5 \text{ ลิตร} = 87.5 \text{ ลิตร}$$

$$\text{ปริมาณชีวภัณฑ์ที่ใช้} = 87.5 \text{ ลิตร} \times \left(\frac{50 \text{ มล.}}{20 \text{ ลิตร}} \right) = 87.5 \times 2.5 = 218.75 \text{ มิลลิลิตร}$$

2. วิธีการพ่นเหมาทั้งแปลงแบบเดิม:

$$\text{ปริมาณน้ำเดิม} = 250 \text{ ต้น} \times 3.5 \text{ ลิตร} = 875 \text{ ลิตร}$$

$$\text{ปริมาณชีวภัณฑ์เดิม} = 875 \times 2.5 = 2,187.5 \text{ มิลลิลิตร}$$

สรุปผลการประหยัด: การใช้ระบบ AI ชี้เป้าช่วยลดการใช้สารชีวภัณฑ์และปริมาณน้ำลงถึง 90% ประหยัดต้นทุนค่ายาและแรงงานได้อย่างมหาศาล

ตัวอย่างการนำไปใช้งานจริงที่ 7.1: ผลกระทบเชิงเศรษฐกิจในวิสาหกิจชุมชนทุเรียนคุณภาพ

วิสาหกิจชุมชนผู้ปลูกทุเรียนแปลงใหญ่ อ.มะขาม จ.จันทบุรี ได้นำแอปพลิเคชันไปทดลองใช้กับสมาชิกจำนวน 30 ครัวเรือน ในรอบฤดูกาลผลิต ปี พ.ศ. 2568–2569
ผลสรุปการประเมินรอบ 6 เดือน:

1. ลดต้นทุนสารเคมีเกษตร: สมาชิกสามารถลดค่าใช้จ่ายสารเคมีและชีวภัณฑ์เฉลี่ย 4,200 บาทต่อไร่

2. รักษาคุณภาพใบก่อนออกดอก: ต้นทุเรียนมีดัชนีพื้นที่ใบสมบูรณ์เฉลี่ยสูงกว่าปีก่อนหน้า 15.2% ทำให้การติดผลสมบูรณ์และลดปัญหาผลแตก/ผลบิดเบี้ยว
3. การสร้างแผนที่ระบาดวิทยาแปลง (Epidemiological Orchard Map): ข้อมูล GPS ที่แอปพลิเคชันบันทึกไว้ช่วยให้ประธานกลุ่มวิสาหกิจสามารถเปิดดูแผนที่ความเสี่ยงบนหน้าจอคอมพิวเตอร์และประกาศเตือนภัยล่วงหน้าแก่สมาชิกในแนวลมได้อย่างแม่นยำ

คำถามทบทวนท้ายบทที่ 7

1. จง อธิบาย โครงสร้าง 4 เลเยอร์ ของ สถาปัตยกรรม แอปพลิเคชัน Mobile Edge AI
2. เหตุใดเฟรมเวิร์ก Flutter จึงเป็นตัวเลือกที่เหมาะสมในการพัฒนาแอปพลิเคชันสำหรับเกษตรกร
3. การออกแบบส่วนต่อประสานผู้ใช้ (UI/UX) สำหรับใช้งานกลางแปลงแดดจ้า ต้องคำนึงถึงปัจจัยใดบ้าง
4. การคำนวณร้อยละความรุนแรงของโรคเชื่อมโยงกับระบบ Prescription Engine อย่างไร
5. จงอธิบายบทบาทของระบบบันทึกพิกัดภูมิศาสตร์ (GPS Geotagging) ในการจัดทำแผนที่โรคระบาด
6. จากตารางผลการประเมิน TAM มิติใดได้รับคะแนนเฉลี่ยสูงสุด และสะท้อนถึงสิ่งใด
7. เพราะเหตุใดการทำงานแบบออฟไลน์ (Offline-First) จึงเป็นหัวใจสำคัญของแอปพลิเคชันเกษตรกรรม

8. จาก ตัวอย่าง การ คำนวณ ที่ 7.1 การ ฉีด พ่น เฉพาะ จุด (Site-Specific Spraying) ส่งผลดีต่อสิ่งแวดล้อมอย่างไร

9. หาก กล้อง สมา ร์ท โฟน ของ เกษตรกร มี เซนเซอร์ ความ ละเอียดต ำมาก ระบบควรมีกลไกแจ้งเตือนอย่างไร

10. จงออกแบบแนวทางการเชื่อมต่อข้อมูลจากสมาร์ทโฟนไปยังระบบคลาวด์ส่วนกลางเมื่อโทรศัพท์กลับมาเชื่อมต่อ Wi-Fi

กิจกรรมฝึกปฏิบัติการประจำบทที่ 7

ปฏิบัติการที่ 7.1: การติดตั้งและทดสอบแอปพลิเคชันสมาร์ทโฟนในแปลงจริง

วัตถุประสงค์: เพื่อ ให้ ผู้ เรียน สามารถ ติด ตั้ง ไฟล์ แอปพลิเคชัน APK บนมือถือและนำไปทดสอบสแกนใบพืชจริง
ขั้นตอนการปฏิบัติ:

1. ถ่าย โอน ไฟล์ `durian_ai_app.apk` ไป ยัง สมา ร์ท โฟน ระบบ ปฏิบัติการ Android
2. ทำการ ติด ตั้ง และ อนุญาต สิทธิ์ การ เข้า ถึง กล้อง และตำแหน่งพิกัด GPS
3. เดิน สำรวจ ต้น ทุเรียน ใน แปลง ทดลอง ทำการ สแกนใบพืช 5 ใบ และตรวจสอบผลการวินิจฉัย บนหน้าจอ
4. ตรวจ สอบ หน้า ประวัติการ บันทึก (History Screen) ว่ามีพิกัดและคำแนะนำการรักษาครบถ้วนหรือไม่

5. สรุป ผล การ ทดสอบ และ ข้อ เสนอ แนะ ใน การ ปรับปรุงความเร็วหรือความแม่นยำของระบบ

Appendix A

คู่มือการติดตั้งและตั้งค่าสภาพแวดล้อมการทำงาน

ภาคผนวกนี้รวบรวมขั้นตอนการติดตั้งและกำหนดค่าสภาพแวดล้อมการทำงาน (Development Environment Setup) สำหรับการพัฒนาและทดสอบระบบปัญญาประดิษฐ์วินิจฉัยโรคใบทุเรียน ทั้งบนระบบคลาวด์ Google Colab และบนเครื่องคอมพิวเตอร์ส่วนบุคคล

A.1 การเตรียมสภาพแวดล้อมบน Google Colab (Cloud GPU)

Google Colab เป็นสภาพแวดล้อมที่เหมาะสมที่สุดสำหรับผู้เริ่มต้นที่ไม่มีการ์ดจอ GPU โดยมีขั้นตอนการตั้งค่าดังนี้:

1. การสร้างสมุดงานและเลือกการ์ดจอ GPU:
    - เข้าสู่เว็บไซต์ `https://colab.research.google.com` และลงชื่อเข้าใช้ด้วยบัญชี Google

- ไปที่เมนู Runtime → Change runtime type
- ในช่อง Hardware accelerator เลือก T4 GPU และกดปุ่ม Save

2. การติดตั้งและอัปเดตแพ็กเกจที่จำเป็น: พิมพ์คำสั่งต่อไปนี้ในเซลล์แรกของสมุดงาน:

```
!pip install --upgrade torch torchvision
    torchaudio
!pip install onnx onnxruntime
    opencv-python matplotlib pillow
    torchinfo
```

3. การตรวจสอบสถานะการเชื่อมต่อ GPU:

```
import torch
print(f"[*] PyTorch Version:
    {torch.__version__}")
print(f"[*] CUDA Available:
    {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"[*] GPU Device Name:
        {torch.cuda.get_device_name(0)}")
```

A.2 การ ติด ตั้ง สภาพ แวดล้อม บน เครื่อง คอมพิวเตอร์ ส่วน บุคคล (Local Machine)

สำหรับ ผู้ พัฒนา ที่ ต้องการ รัน ระบบ บน เครื่อง คอมพิวเตอร์ส่วนบุคคล (รองรับทั้ง Windows, macOS และ Linux):

1. การสร้างสภาพแวดล้อมเสมือน (Python Virtual Environment):

```
#
python3 -m venv venv_durian

#             (macOS/Linux)
source venv_durian/bin/activate

#             (Windows PowerShell)
.\venv_durian\Scripts\Activate.ps1
```

2. การติดตั้งไลบรารีจากไฟล์ requirements.txt:

```
pip install -r requirements.txt
```

A.3 การติดตั้งชุดพัฒนา Flutter สำหรับการสร้างแอปพลิเคชันมือถือ

การสร้างแอปพลิเคชันสมาร์ทโฟนสำหรับติดตั้งบนระบบปฏิบัติการ Android และ iOS มีขั้นตอนดังนี้:

1. ดาวน์โหลด Flutter SDK เวอร์ชัน ล่าสุด จาก เว็บไซต์ทางการ `https://flutter.dev`
2. แตกไฟล์และเพิ่มพาธของโฟลเดอร์ `flutter/bin` เข้าสู่ System Environment Variables (PATH)
3. เปิดเทอร์มินัล และรันคำสั่งตรวจสอบความพร้อม: `flutter doctor`
4. ติดตั้งปลั๊กอิน สำหรับ ONNX Runtime ใน ไฟล์ `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.10.5
```

```
  onnxruntime: ^1.2.0
  path_provider: ^2.1.2
```

Appendix B

ตารางค่าพารามิเตอร์ของแบบจำลองและการปรับแต่ง

ภาค ผนวก นี้ รวบรวม ราย ละเอียด พารามิเตอร์ ทั้งหมด ของ การ ฝึกฝน โมเดล MobileNetV3-Small โครงสร้างเลเยอร์ และการกำหนดค่าในระบบ Explainable AI เพื่อให้สามารถทำซ้ำผลการทดลอง (Reproducibility) ได้อย่างสมบูรณ์

B.1 ตารางค่าไฮเปอร์พารามิเตอร์สำหรับการฝึกฝนโมเดล (Training Configurations)

ตารางที่ B.1 รายละเอียดการตั้งค่าไฮเปอร์พารามิเตอร์สำหรับการฝึกฝนโมเดล MobileNetV3-Small

ชื่อพารามิเตอร์	ค่าที่กำหนดใช้งาน	คำอธิบายเหตุผลทางเทคนิค
Image Resolution	$224 \times 224 \times 3$	ขนาดมาตรฐานที่สมดุลระหว่างความละเอียดและขนาดเทนเซอร์
Batch Size	32	ขนาดบัตช์ที่เหมาะสมกับหน่วยความจำ GPU และ

B.2 ตาราง โครงสร้าง เลเยอร์ ของ โมเดล MobileNetV3-Small

ตารางที่ B.2 โครงสร้างสถาปัตยกรรมภายในและขนาด Feature Map ของ MobileNetV3-Small

บล็อก / เลเยอร์	ขนาด Input	เคอร์เนล / ขยาย	SE Module	ขนาด Output
Conv2d 3x3	$224 \times 224 \times 3$	Stride 2	ไม่ใช้	$112 \times 112 \times 16$
bneck (Inverted)	$112 \times 112 \times 16$	k3 / exp 16	มี (SE)	$56 \times 56 \times 16$
bneck	$56 \times 56 \times 16$	k3 / exp 72	ไม่ใช้	$28 \times 28 \times 24$
bneck	$28 \times 28 \times 24$	k3 / exp 88	ไม่ใช้	$28 \times 28 \times 24$
bneck	$28 \times 28 \times 24$	k5 / exp 96	มี (SE)	$14 \times 14 \times 40$
bneck	$14 \times 14 \times 40$	k5 / exp 240	มี (SE)	$14 \times 14 \times 40$
bneck	$14 \times 14 \times 40$	k5 / exp 120	มี (SE)	$14 \times 14 \times 48$
bneck	$14 \times 14 \times 48$	k5 / exp 144	มี (SE)	$14 \times 14 \times 48$
bneck	$14 \times 14 \times 48$	k5 / exp 288	มี (SE)	$7 \times 7 \times 96$
bneck	$7 \times 7 \times 96$	k5 / exp 576	มี (SE)	$7 \times 7 \times 96$
Conv2d 1x1	$7 \times 7 \times 96$	Stride 1	ไม่ใช้	$7 \times 7 \times 576$
AdaptiveAvgPool	$7 \times 7 \times 576$	Global Pool	-	$1 \times 1 \times 576$
Linear Head 1	$1 \times 1 \times 576$	Hard-Swish	-	1024
Linear Head 2	1024	Dropout 0.3	-	4 (คลาสโรค)

Appendix C

พจนานุกรมคำศัพท์เทคนิค และนิยามศัพท์

ภาคผนวกนี้รวบรวมคำศัพท์เทคนิคเฉพาะทางด้านคอมพิวเตอร์วิทัศน์ การเรียนรู้เชิงลึก พยาธิวิทยาพืช และเกษตรแม่นยำ พร้อมคำนิยามและความหมายภาษาไทย

Backpropagation (การแพร่ย้อนกลับ)
: กระบวนการ คำนวณ อนุพันธ์ ย่อย ของ ฟังก์ชัน การ สูญ เสีย เทียบ กับ ค่าน้ำ หนัก ทุก ตัว ใน โครง ข่าย ประสาท เทียม เพื่อ ใช้ ใน การ ปรับปรุง ค่าน้ำหนัก ผ่าน ขั้น ตอน วิธี Gradient Descent

Batch Normalization (การปรับมาตรฐานบัตช์)
: เทคนิคการปรับค่าข้อมูลในมินิบัตช์ ให้มีค่าเฉลี่ยเป็นศูนย์และส่วนเบี่ยงเบนมาตรฐานเป็นหนึ่ง เพื่อช่วยลด

ปัญหา Internal Covariate Shift และทำให้โมเดลลู่เข้าเร็วขึ้น

Bounding Box (กรอบสี่เหลี่ยมระบุพิกัด)

พิกัดสี่เหลี่ยม $(x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}})$ ที่ใช้ระบุตำแหน่งและขอบเขตของวัตถุหรือรอยโรคบนภาพดิจิทัล

Confusion Matrix (เมทริกซ์ความสับสน)

ตารางสรุปผลการทำนายของโมเดลเทียบกับค่าความจริง ซึ่งประกอบด้วยช่อง True Positive, False Positive, True Negative และ False Negative

Depthwise Separable Convolution (การแยกมิติคอนโวลูชันเชิงลึก)

เทคนิคการแยกการคำนวณคอนโวลูชันออกเป็น Depthwise Conv (กรองเชิงพื้นที่) และ Pointwise Conv (รวมช่องสัญญาณ) เพื่อลดปริมาณการคำนวณลง 8–9 เท่า

Early Stopping (การหยุดก่อนกำหนด)

เทคนิค Regularization ที่ติดตามค่า Validation Loss และยุติการฝึกฝนทันทีเมื่อผลการประเมินเริ่มแย่ลงต่อเนื่องตามจำนวนรอบที่กำหนด เพื่อป้องกัน Overfitting

Edge Computing (การประมวลผลบนขอบ)

สถาปัตยกรรมการประมวลผลข้อมูลที่ดำเนินการบนอุปกรณ์ปลายทาง

โดยตรง เช่น สมาร์ทโฟน หรือบอร์ดสมอง กล ฝัง ตัว โดย ไม่ ต้อง พึ่งพา เซิร์ฟเวอร์คลาวด์

Explainable AI - XAI (ปัญญาประดิษฐ์อธิบายได้)

กลุ่มเทคนิคทางปัญญาประดิษฐ์ที่มุ่งเน้น การ เปิด เผย เหตุผล กลไก หรือพื้นที่ที่โมเดลใช้ในการตัดสินใจ เพื่อสร้างความโปร่งใสและความน่าเชื่อถือให้แก่มนุษย์

Focal Loss (ฟังก์ชันการสูญเสียแบบโฟคอล)

ฟังก์ชัน Loss ที่ปรับปรุงจาก Cross-Entropy โดยเพิ่มตัวคูณถ่วงน้ำหนัก เพื่อ ลด ความ สำคัญ ของ ตัวอย่าง ที่จำแนกได้ง่าย และเพิ่มความสำคัญให้กับตัวอย่างที่จำแนกได้ยาก

Grad-CAM (การสร้างแผนที่ความร้อนถ่วงน้ำหนักเกรเดียนต์)

เทคนิค XAI ที่ ใช้ เก ร เดีย นต์ ของ คลาส เป้า หมาย เทียบ กับ เลเยอร์ คอน โว ลู ชัน สุดท้าย เพื่อ สร้าง ภาพ ความร้อนระบุบริเวณที่ส่งผลต่อการตัดสินใจ

Inverted Residuals (โครงสร้างตกค้างแบบกลับด้าน)

บล็อก โครงสร้าง ใน MobileNetV2/V3 ที่ทำการขยายช่องสัญญาณก่อน ทำการคอนโวลูชันเชิงลึก จากนั้นจึงบีบอัดกลับลงมา เพื่อรักษาข้อมูลไม่ให้สูญหาย

Mycelium (เส้นใยรา) โครงสร้างเส้นสายของเชื้อราที่เจริญแทรกซึมเข้าสู่เนื้อเยื่อพืชเพื่อดูดซับสาร อาหาร และ ปล่อย เอนไซม์ ย่อยสลายเซลล์

ONNX - Open Neural Network Exchange
มาตรฐาน เปิด สำหรับ แลก เปลี่ยน และจัดเก็บโมเดลโครงข่ายประสาทเทียม ช่วยให้สามารถถ่ายโอนโมเดลข้าม เฟรม เวิร์ก และ รัน บน เอน จินต่างๆ ได้อย่างอิสระ

Quantization (การลดทอนความแม่นยำตัวเลข)
กระบวนการ แปลง ค่า พารามิเตอร์ของโมเดลจากจำนวนจริง Floating-point (FP32) ไป เป็น จำนวนเต็ม Integer (INT8) เพื่อ ลด ขนาด ไฟล์ และเพิ่มความเร็วในการประมวลผล

Squeeze-and-Excitation - SE Module
กลไก ความ ใส่ใจ ที่ คำนวณ และ ปรับ ค่าน้ำ หนัก ความ สำคัญ ของ แต่ละช่องสัญญาณ (Channel-wise Attention) โดยอัตโนมัติ

Transfer Learning (การเรียนรู้แบบถ่ายโอน)
กระบวนการ นำ โครง ข่าย ประสาทเทียมที่ผ่านการฝึกฝนบนชุดข้อมูลขนาดใหญ่มา ปรับ ใช้ กับ งาน ใหม่ ที่มีข้อมูล จำกัด โดย คง ค่าน้ำ หนัก ตัวสกัดคุณลักษณะเดิมไว้

บรรณานุกรม

[ภาษาไทย / Thai References]

กรมวิชาการเกษตร. (2565). คู่มือการวินิจฉัยและป้องกันกำจัดโรคและแมลงศัตรูทุเรียนที่สำคัญในเขตภาคตะวันออก. กรุงเทพฯ: สำนักวิจัยและพัฒนาการอารักขาพืช กระทรวงเกษตรและสหกรณ์.

จันทมาลี, จ., และ ทัศนา, ช. (2567). การศึกษาสัณฐานวิทยา และ การ แพร่ กระจาย ของ เชื้อ สาหร่าย Cephaleuros virescens บนใบทุเรียนพันธุ์หมอนทองใน จังหวัด จันทบุรี. วารสาร วิทยาศาสตร์ และ นวัตกรรมการเกษตร, 8(2), 145–158.

ทัศนา, ช., และ จันทมาลี, จ. (2568). สถาปัตยกรรมไอโอทีและการเรียนรู้เชิงลึกเพื่อการเกษตรแม่นยำในแปลงสวน ทุเรียน อัจฉริยะ. จันทบุรี: คณะ วิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี.

ทัศนา, ช., สุข สมบูรณ์, ภ., และ วงศ์สุวรรณ, ว. (2568). การพัฒนาระบบประมวลผลบนอุปกรณ์ขอบด้วยเทคนิค Explainable AI สำหรับตรวจจับโรคพืชผลไม้เมืองร้อน. วารสารวิจัยและพัฒนา มหาวิทยาลัยราชภัฏรำไพพรรณี, 19(1), 88–102.

สำนักงานเศรษฐกิจการเกษตร. (2567). รายงานสถิติการค้าสินค้าเกษตรไทยกับต่างประเทศ ประจำปี 2566. กรุงเทพฯ: ศูนย์ สารสนเทศ การเกษตร กระทรวงเกษตรและสหกรณ์.

[ภาษาอังกฤษ / International English References (APA 7th Edition)]

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V., & Adam, H. (2019). Searching for MobileNetV3. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (pp. 1314–1324). https://doi.org/10.1109/ICCV.2019.00140

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861. https://doi.org/10.48550/arXiv.1704.04861

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. In Proceedings of the IEEE Conference on Computer Vision and

Pattern Recognition (CVPR) (pp. 7132–7141). https://doi.org/10.1109/CVPR.2018.00745

Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision (ICCV) (pp. 2980–2988). https:// doi.org/ 10.1109/ ICCV.2017.324

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In International Conference on Learning Representations (ICLR). https://doi.org/10.48550/arXiv.1711.05101

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, Article 1419. https:// doi.org/ 10.3389/ fpls.2016.01419

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 4510–4520). https:// doi.org/ 10.1109/ CVPR.2018.00474

Selvaraju, R. R., Cogswell, M., Das, A., Vedaldi, A., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE International Conference on Com-

puter Vision (ICCV) (pp. 618–626). https://doi.org/10.1109/ICCV.2017.74

Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning (ICML) (pp. 6105–6114). PMLR. https:// proceedings.mlr.press/ v97/ tan19a.html

Thassana, C., & Janthamalee, J. (2025). Explainable artificial intelligence with MobileNetV3 architecture for real-time plant disease detection on smart edge devices. Computers and Electronics in Agriculture, 225, Article 109280. https:// doi.org/ 10.1016/ j.compag.2025.109280

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). mixup: Beyond empirical risk minimization. In International Conference on Learning Representations (ICLR). https://doi.org/10.48550/arXiv.1710.09412

Index

AdamW Optimizer, 53

Cephaleuros virescens, 6
Classifier Head, 41
Colletotrichum gloeosporioides, 6
Confusion Matrix, 49, 55
Convolutional Neural Networks, 33
Cross-Entropy Loss, 49, 53

Data Augmentation, 17
DataLoader, 25
Depthwise Separable Convolution, 37

Edge AI, 1, 83
Explainable AI, 65, 69

Flutter Framework, 99
Flutter SDK, 117

Geometric Augmentation, 22
Google Colab, 115
Grad-CAM, 65

HUD Scanner, 104
Hyperparameters, 119

InferenceController, 107
Inverted Residuals, 37

Kernel, 37

Learning Curves, 54

MobileNetV3, 33

ONNX Export, 83
ONNX Runtime, 87
Overfitting, 54

Photometric Augmentation, 22
Precision & Recall, 55
PyTorch Dataset, 25
PyTorch Training, 49
PyTorch Training Loop, 57

Quantization, 83
Quantization INT8, 88

Rhizoctonia solani, 6

Squeeze-and-Excitation, 39

Terminal Execution, 56
TorchScript, 87
Transfer Learning, 33, 41

กล้องสดเรียลไทม์, 103
การก ระ จา ย ตัว ของ ข้อมูล, 21
การ ติด ตั้ง สภาพ แวดล้อม, 115
การ ประมวล ผล ภาพ ดิจิทัล, 17
การ ยอมรับ ของ เกษตรกร, 105
การยอมรับเทคโนโลยี, 99
การเปรียบเทียบโมเดล, 40
ข้อจำกัดฮาร์ดแวร์, 87
ความรุนแรงของโรค, 65
คอนโวลูชัน, 37
ชุดข้อมูลภาพใบทุเรียน, 17
ทุเรียนหมอนทอง, 4
นิยามศัพท์, 123
ปฏิทินการระบาด, 8
ปริภูมิสี HSV, 20
ปริภูมิสี RGB, 20
พจนานุกรมคำศัพท์, 123
พยาธิวิทยาพืช, 4
พารามิเตอร์ แบบ จำลอง, 119
ภาพ ความ ร้อน Grad-CAM, 73
ระดับ ความ รุนแรง ของ โรค, 74
ระบบโครงข่ายประสาท, 7
ระบาดวิทยาตามฤดูกาล, 8
สคริปต์ Python, 11
สถาปัตยกรรม Edge AI, 7
สถาปัตยกรรม Flutter, 103
สภาพแวดล้อมแปลงเปิด, 10
สมการ Grad-CAM, 71
ส่วนต่อประสานผู้ใช้, 104
อุปกรณ์ขอบ, 87
เกณฑ์ มาตรฐาน ฮาร์ดแวร์, 90
เกษตรอัจฉริยะ, 1
แบบจำลอง TAM, 105
แอป พลิ เค ชัน สมา ร์ท โฟน, 99
โค้ด Flutter, 107
โค้ด GradCAM, 76
โค้ดส่งออก ONNX, 91
โมเดลกล่องดำ, 69
โรคใบทุเรียน, 1

ประวัติผู้เขียน



ผู้ ช่วย ศาสตราจารย์ ดร.ชีวะ ทัศนา

ตำแหน่ง ทาง วิชาการ: ผู้ ช่วย ศาสตราจารย์
สาขาวิชาฟิสิกส์

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัย ราชภัฏรำไพพรรณี

อีเมล: chewathassana@rbru.ac.th

ประวัติการศึกษา

- ปร.ด. (ฟิสิกส์ประยุกต์)
  สถาบัน เทคโนโลยี พระจอมเกล้า เจ้าคุณทหารลาดกระบัง
- วท.ม. (ฟิสิกส์ประยุกต์)
  สถาบัน เทคโนโลยี พระจอมเกล้า เจ้าคุณทหารลาดกระบัง
- วท.บ. ฟิสิกส์ (คอมพิวเตอร์และอิเล็กทรอนิกส์)
  มหาวิทยาลัยนเรศวร

ความเชี่ยวชาญและงานวิจัย

- ฟิสิกส์เกษตรดิจิทัลและไอโอที (AIoT SmartFarm)
- ปัญญาประดิษฐ์และการเรียนรู้เชิงลึก (Computer Vision)
- การ คำนวณ โครงสร้าง สาร แม่เหล็ก และควอนตัมฟิสิกส์